Introduction to Chemometrics

Espectrometrika Academy — Learner's Guide

Learning path · Preprocessing · Exploration · Class modeling · Predictive modeling · Validation · Metrics · Hyperparameters · Public datasets

Companion reference for Espectrometrika Academy, the educational edition: learn chemometrics step by step on spectral or tabular data, and use the same tools to predict new samples with full traceability. Read it alongside your analysis to understand what each tool does, how to read its plots and metrics, and which setting to try next.

Table of Contents

Table of Contents 2
1. What is chemometrics? 4
2. Signal preprocessing 4
2.1 Centering and scaling (universal) 4
Mean centering 4
Autoscaling / Standardization 4
2.2 NIR / MIR: scatter correction 5
SNV (Standard Normal Variate) 5
MSC (Multiplicative Scatter Correction) 5
2.3 EMSC (Extended Multiplicative Scatter Correction) 6
2.4 Smoothing and derivatives (Savitzky-Golay) 7
Smoothing (0th-order derivative) 7
1st derivative 7
2nd derivative 7
2.5 Raman 8
2.6 Chromatograms 8
2.7 NMR 9
3. Exploratory Data Analysis (EDA) 10
3.1 PCA (Principal Component Analysis) 10
Biplot 10
Loading plot 11
Reading loadings and the biplot step by step 11
3.2 Outlier detection: Hotelling's T^2 and residual Q 12
3.3 HCA — Hierarchical Cluster Analysis 13
3.4 t-SNE and UMAP 14
3.5 MCR-ALS (Multivariate Curve Resolution — Alternating Least Squares) 14
4. Variable selection 16
4.1 Boruta 16
4.2 Genetic Algorithm 16
5. Class modeling: SIMCA 17
5.1 How each class model is built 17
5.2 Why the Q limit must be estimated by cross-validation 18
5.3 Acceptance criterion and per-class metrics 18
6. Predictive modeling 19
6.1 Classification 19
6.2 Regression 19
6.3 Illustrative diagrams for selected algorithms 20
LDA — linear separation 20
SVM — maximum margin 21
Decision Tree 21
Random Forest — voting across many trees 22
XGBoost — sequential boosting 22
KNN — nearest neighbors 22
Neural Network (MLP) 23
Lasso, Ridge and Elastic Net — regularization 23
7. Train/test split and cross-validation 24
7.1 Independent test set 24
Random split vs. Kennard-Stone 24

7.2 Cross-validation (k-fold and Leave-One-Out)..... 24
8. Performance metrics..... 26
8.1 Classification..... 26
ROC curve and AUC..... 27
8.2 Regression..... 27
9. Model diagnostics..... 29
9.1 Leverage / Williams plot..... 29
9.2 Learning curves..... 30
9.3 Statistical comparison between models..... 31
10. Hyperparameter optimization..... 33
10.1 Search strategies..... 33
10.2 The app's speed/quality presets..... 33
11. How far can a prediction be trusted?..... 34
11.1 Conformal prediction..... 34
11.2 Applicability domain and data-driven SIMCA..... 35
11.3 Why did the model say that? Explaining one prediction..... 36
12. From bands to chemistry, and tabular data..... 37
12.1 Functional-group assignment..... 37
12.2 Tabular data and metabolomics mode..... 38
13. Your learning path in the app..... 38
13.1 The three guide buttons..... 39
13.2 Observation boxes and the final report..... 39
14. Metric cheat sheet..... 39
14.1 Classification..... 39
14.2 Regression..... 40
14.3 SIMCA..... 41
14.4 Outliers (T^2 and Q)..... 41
14.5 PCA..... 42
14.6 Dendrogram..... 42
14.7 Prediction..... 43
15. Hyperparameters: defaults, data-based ranges and your own..... 43
15.1 Rules used for the recommended ranges..... 44
15.2 Good practice when editing ranges..... 44
16. Public datasets to practise (and how to cite them)..... 45
Putting it all together..... 48
Contact..... 48

1. What is chemometrics?

Chemometrics applies mathematical and statistical methods to extract chemically relevant information from experimental data — typically spectra (NIR, MIR, Raman, NMR), chromatograms, or other instrumental signals with many variables. Instead of reading a spectrum peak by peak, the full signal is analyzed at once to find patterns, group similar samples, detect anomalies, or predict a property of interest (a concentration, a product class, an origin) from the overall shape of the signal.

A typical chemometrics workflow follows these stages, which also map onto the tabs of the companion app:



> **Key idea**
>
> Every stage depends on the previous one: bad preprocessing ruins PCA; a misread PCA lets outliers slip through; and undetected outliers distort any predictive model built afterward. That is why the order matters, and why this guide follows the same order as the app's tabs.

2. Signal preprocessing

No instrumental signal comes out of the instrument "clean": there is always noise, baseline shifts, or physical measurement effects (light scattering, dilution, alignment) that have nothing to do with the sample's chemistry but still affect the measured signal. Preprocessing aims to correct these unwanted effects without erasing the real chemical information.

2.1 Centering and scaling (universal)

These two transformations are applied to virtually any type of data, as a preliminary step before PCA, PLS, or any other multivariate method — regardless of the type of spectrum.

Mean centering

$$x'ij = xij - \bar{x}j$$

For each variable (column) j, the average of that variable across all samples is subtracted. This shifts the origin of the data to the center of the point cloud, so the analysis focuses on the differences between samples (the variance) rather than the absolute signal level. Corrects for: the baseline level common to all samples.

Autoscaling / Standardization

$$x'ij = (xij - \bar{x}j) / sj$$

In addition to centering, each variable is divided by its own standard deviation (computed across samples). The result: every variable ends up with variance 1, carrying the same "weight" in the analysis. Used when variables have very different scales or units, so that zones of higher absolute intensity don't dominate the analysis just because of their magnitude. Corrects for: differences in scale between variables.

2.2 NIR / MIR: scatter correction

In reflectance spectroscopy (typical for NIR on solids and powders), differences in particle size or sample packing create a light-scattering effect that overlays the chemical signal: each spectrum ends up with its own offset and scale, unrelated to chemical composition.

SNV (Standard Normal Variate)

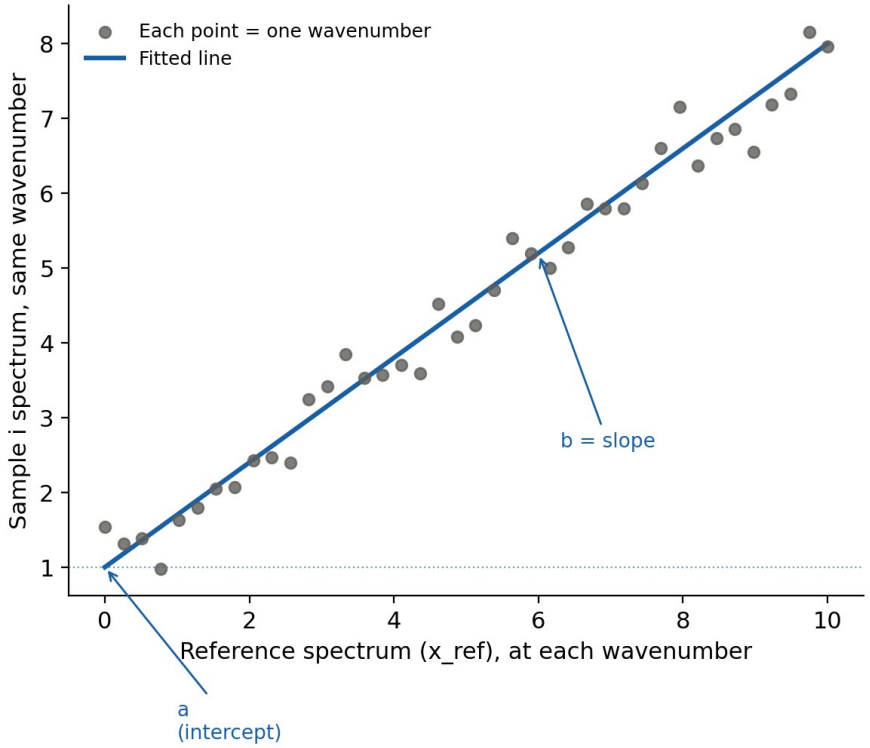
$$x'i = (xi - \bar{x}i) / si$$

For each spectrum (row) i, its own mean is subtracted and it is divided by its own standard deviation, computed across the whole spectrum. Each sample corrects its own scatter independently. Corrects for: light scattering from particle size and optical path differences.

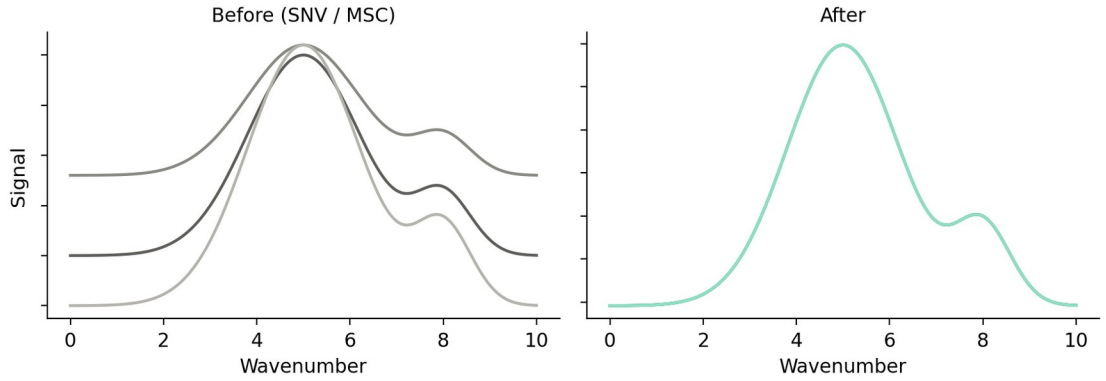
MSC (Multiplicative Scatter Correction)

$$xi = ai + bi \cdot x_ref \quad -> \quad x'i = (xi - ai) / bi$$

Unlike SNV, MSC fits each spectrum against a common reference spectrum (usually the average of all calibration samples) via linear regression. Each point in the plot below is a different wavenumber: on the X axis, how much the reference absorbs there; on the Y axis, how much that sample absorbs at the same point. Without scatter, every point would fall on the ideal diagonal.



a = additive offset · b = scale difference. The correction undoes both effects.



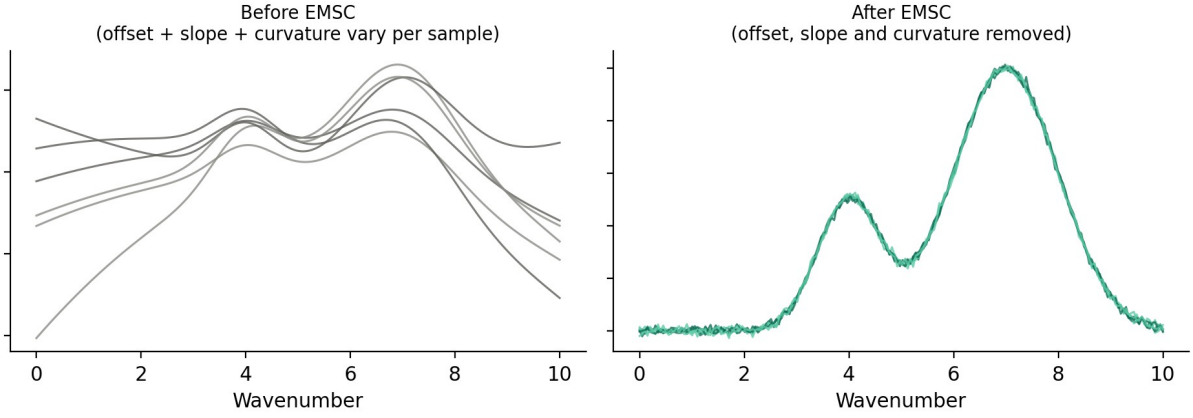
Typical SNV/MSC effect: spectra with different offset and slope collapse to a common shape.

2.3 EMSC (Extended Multiplicative Scatter Correction)

EMSC extends plain MSC by also modeling and removing wavelength-dependent effects — a polynomial in the (normalized) wavelength axis, on top of the usual additive offset and multiplicative scaling. This corrects sloped or curved baseline effects that plain MSC cannot, because MSC assumes the scatter effect is the same at every wavelength.

$$xi(\lambda) = ai + bi \cdot x_ref(\lambda) + ci \cdot p1(\lambda) + di \cdot p2(\lambda) + \ldots + ei(\lambda)$$

The correction fits this model per sample by least squares and then removes everything except the bi·x_ref term: x'i(λ) = [xi(λ) - ai - ci·p1(λ) - di·p2(λ) - …] / bi. A polynomial order of 0 behaves like plain MSC; order 2 (the app's default) also removes linear and quadratic baseline slopes, which is the most common real-world scenario.



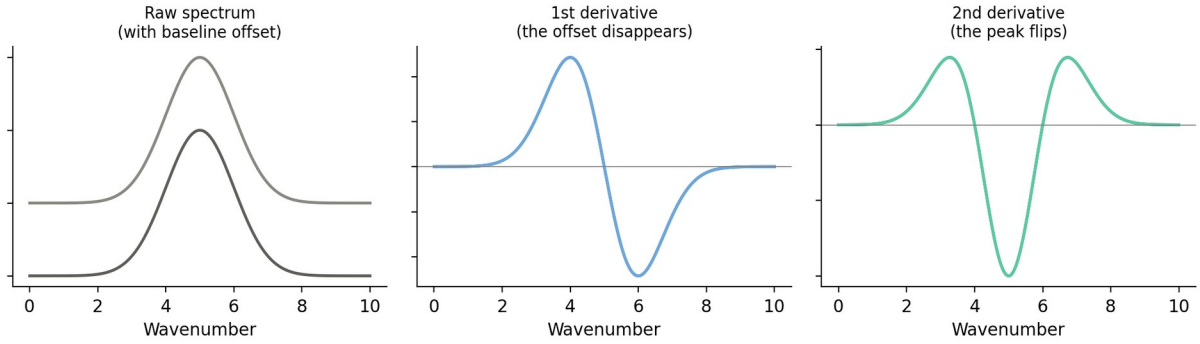
EMSC removes offset, slope AND curvature at once — notice how much flatter and more overlapping the 'after' spectra are compared to plain MSC.

When to prefer EMSC over MSC

If your spectra show scatter effects that clearly depend on wavelength (e.g. a curved or sloped baseline that isn't the same shape across samples), EMSC will usually remove more of that nuisance variation than plain MSC — at the cost of one extra parameter (the polynomial order) to choose. Too high an order risks removing real chemical signal along with the scatter, so start at order 1-2 and compare results.

2.4 Smoothing and derivatives (Savitzky-Golay)

The Savitzky-Golay filter fits a low-order polynomial by least squares within a moving window of points, and replaces the central point with the value of that polynomial (or its derivative). Unlike a simple moving average, it preserves peak shape and width much better.



Smoothing (0th-order derivative)

Corrects for: high-frequency, random instrumental noise.

1st derivative

$$dx / d\lambda$$

Highlights slopes and changes in the spectrum, improving the resolution of overlapping bands. Corrects for: constant (additive) baseline offsets — a spectrum shifted up or down has the same derivative as the original.

2nd derivative

$$d^2x / d\lambda^2$$

Flips peaks (they appear as valleys) and further improves resolution of heavily overlapping bands — widely used for band assignment. Corrects for: constant and sloped (additive and linear) baseline.

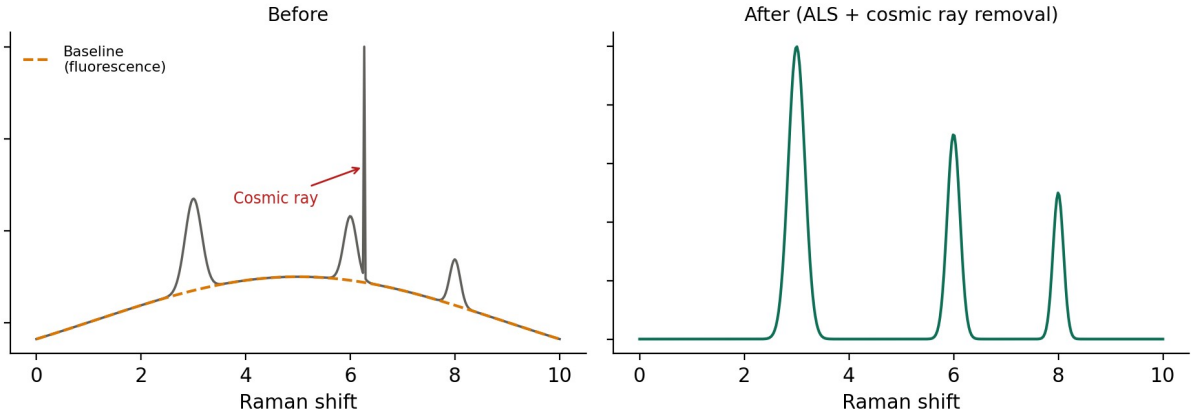
Practical rule

Derivatives amplify high-frequency noise substantially. That is why they are almost always smoothed before differentiating (the Savitzky-Golay filter itself does both in a single pass, given the right window and polynomial order).

2.5 Raman

Raman shares Savitzky-Golay smoothing and derivatives with NIR/MIR, but has two problems of its own that usually dominate its preprocessing:

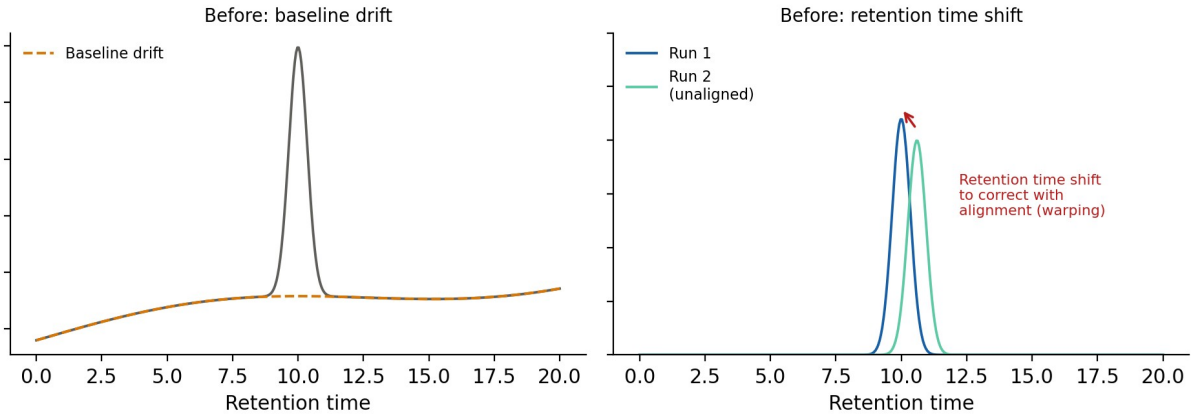
- **Background fluorescence:** a wide, curved baseline, often more intense than the Raman signal itself. Corrected with baseline-fitting methods such as ALS (Asymmetric Least Squares), which fits a smooth curve giving less weight to points above it (the peaks) and subtracts it.
- **Cosmic rays:** narrow, anomalously intense spikes — a CCD detector artifact unrelated to the sample. Detected by comparing each point against a local median filter, and the outlier values replaced.



Left: background fluorescence and a cosmic ray. Right: corrected spectrum.

2.6 Chromatograms

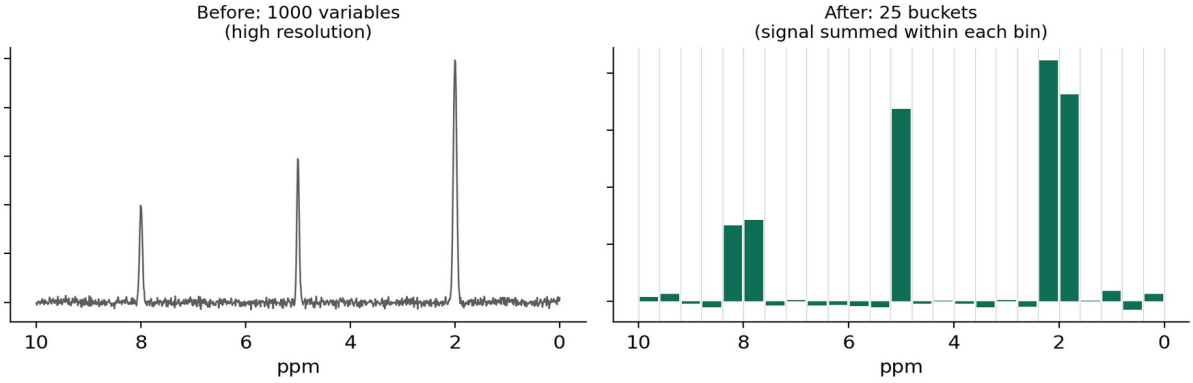
- **Baseline correction:** same concept as in Raman (ALS or other fits), to correct instrumental drift or column bleed.
- **Peak alignment (warping):** techniques such as COW (Correlation Optimized Warping) warp each run's time axis to correct retention-time shifts (from column aging, flow or temperature variations), needed to compare multiple runs against each other.
- **Normalization:** by total area or against an internal standard, to correct for differences in injected volume or sample dilution.



Two typical issues: baseline drift (left) and retention-time shift between runs (right).

2.7 NMR

- **Bucketing / binning:** divides the spectrum into fixed-width "buckets" (e.g. 0.04 ppm) and sums the signal within each, drastically reducing the number of variables. Corrects for: small peak shifts between samples (integrating within a window makes a small shift within that window a non-issue).
- **PQN (Probabilistic Quotient Normalization):** computes a reference spectrum (the median of all samples) and, for each sample, uses the median of the variable-by-variable quotients against that reference as the normalization factor. Corrects for: overall sample dilution differences, more robustly than normalizing by total area.



Bucketing: the high-resolution spectrum is grouped into bins, summing the signal within each.

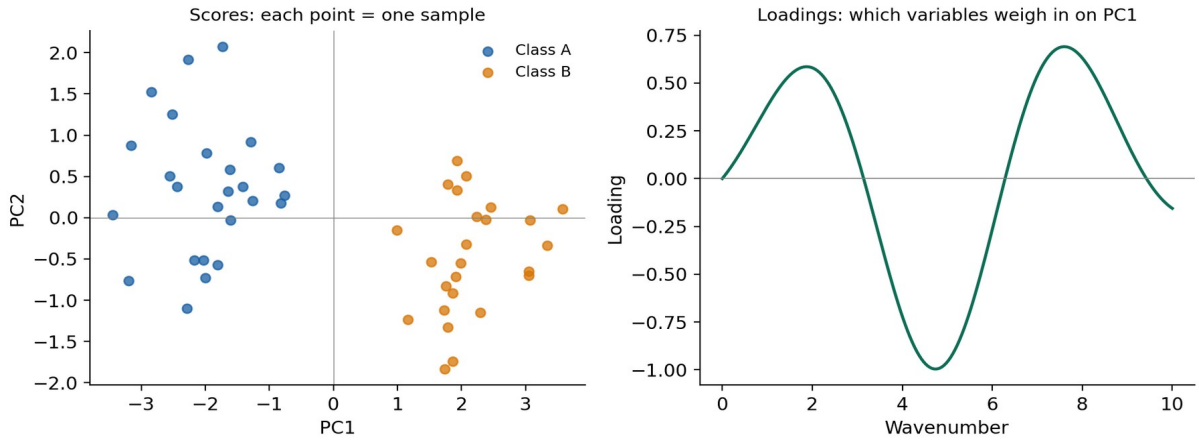
3. Exploratory Data Analysis (EDA)

Before building any predictive model, it helps to "look" at the overall structure of the data: are there natural groupings? Are there unusual samples (outliers) that could distort a model? Exploratory analysis answers these questions without needing a reference variable (class or value to predict).

3.1 PCA (Principal Component Analysis)

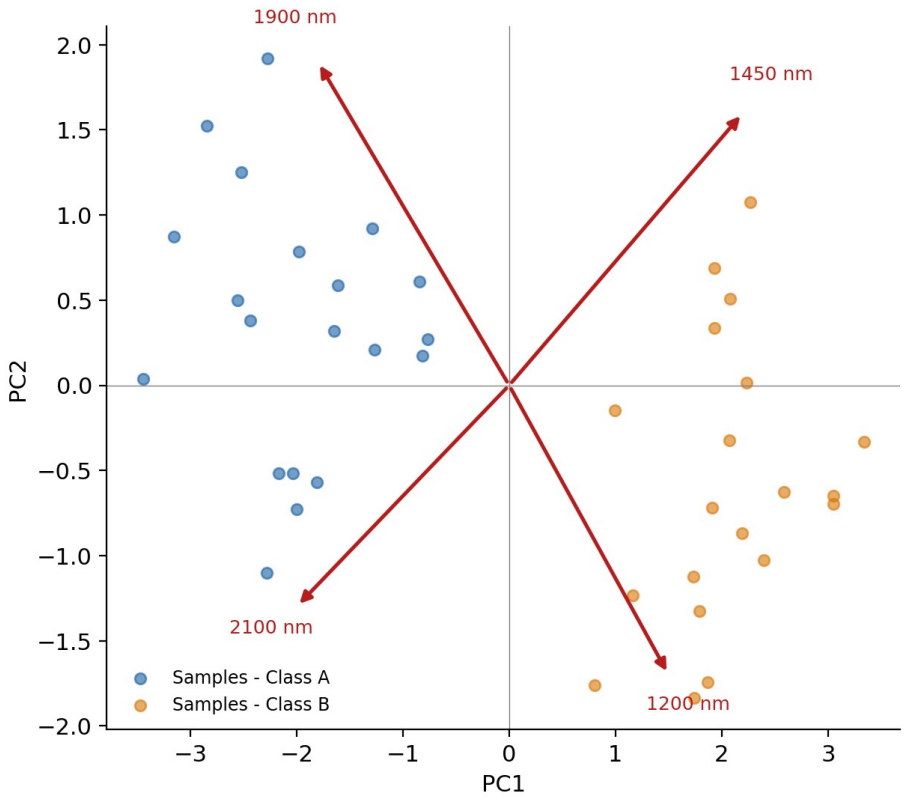
PCA summarizes hundreds of correlated variables (the wavenumbers of a spectrum) into a few "principal components" (PCs) that capture most of the data's variability, in the order in which they explain it. Each component is a linear combination of the original variables.

- **Scores:** each sample's position in the space of the principal components. Chemically similar samples end up close together; different ones, far apart. This is what gets plotted as PC1 vs PC2, etc.
- **Loadings:** how much each original variable (each wavenumber) weighs in on each component. They allow chemical interpretation of which spectral regions explain the observed variability.
- **Explained variance:** the scree plot shows what percentage of the total variability each component captures, helping decide how many are relevant.



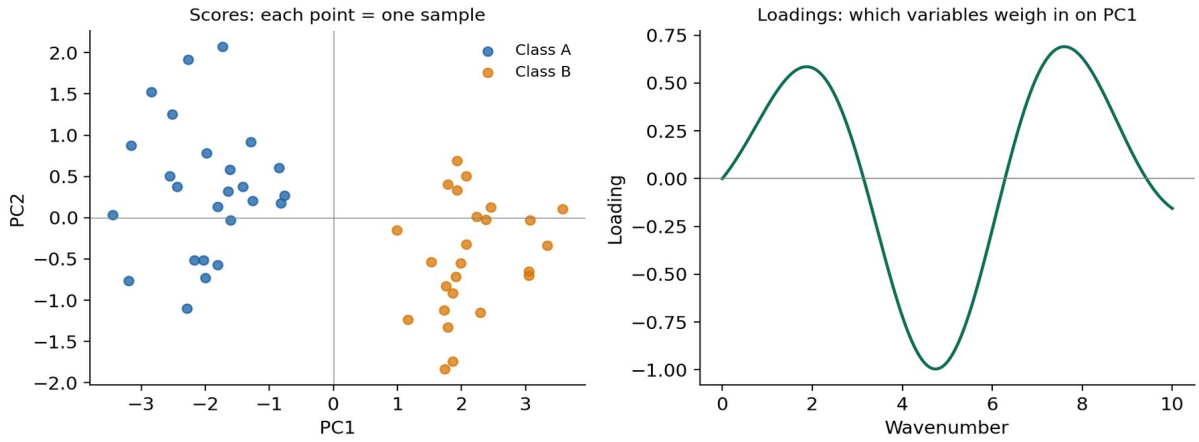
Biplot

Combines scores (the samples) and loadings (the variables) in the same plot, shown as arrows from the origin. The direction and length of each arrow indicates how strongly, and in what direction, that variable influences the plotted components — useful for visually connecting "which samples cluster together" with "which wavenumbers explain it".



Loading plot

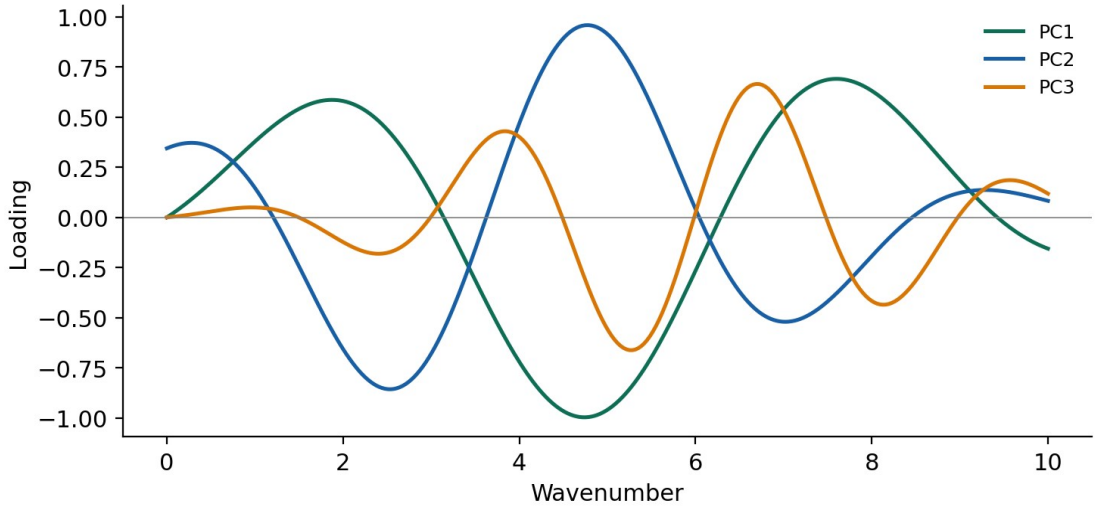
Shows the full loading curve of one or more components across the entire spectrum (instead of a single vector, as in the biplot). Lets you pinpoint exactly which wavenumbers each component weighs most heavily, for chemical assignment.



Reading loadings and the biplot step by step

A loading is the weight of one original variable in one component. A large positive or negative loading means that the variable contributes strongly to that PC; a value near zero means it hardly takes part. Loadings carry the chemistry: scores tell you WHICH samples differ, loadings tell you WHICH variables make them differ.

- **Spectra:** plot the loading of PC1 against wavelength. Peaks and valleys point to the bands that change between your samples. Positive and negative lobes are two sides of the same contrast: samples with positive scores have more of the positive-loading bands and less of the negative ones.
- **Tables:** the biplot draws every variable as an arrow. Arrows pointing in the same direction are positively correlated, opposite arrows are negatively correlated, and arrows at 90° are roughly uncorrelated. A long arrow means the variable is well represented in that plane.
- A group of samples lying in the direction of an arrow has high values of that variable; samples on the opposite side have low values.
- **Caution:** the arrows are only reliable for the variance shown in the plane. Check the explained variance of the two PCs (printed under the plot) before interpreting short arrows.
- Loadings change sign arbitrarily between runs (a PC and its mirror image are equivalent). Interpret contrasts, not signs.



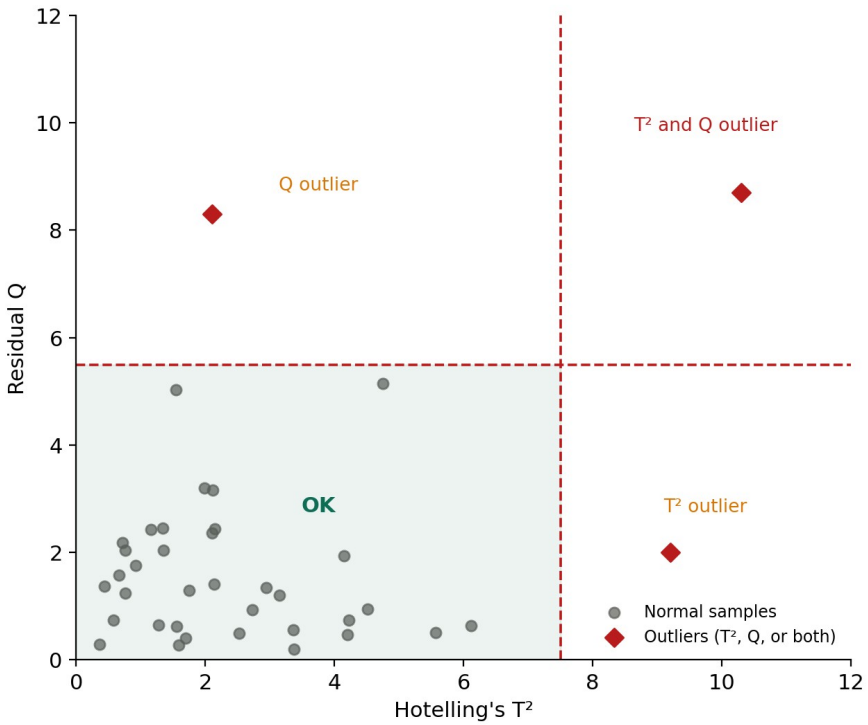
3.2 Outlier detection: Hotelling's T^2 and residual Q

Two complementary statistics are used, computed over a chosen number of components:

- **Hotelling's T^2:** measures how far a sample is from the center of the model, within the space defined by the retained components (distance WITHIN the model).
- **Residual Q (SPE):** measures how well each sample is described by the model — the portion of variability that the retained components do NOT capture (distance TO the model).

> $T^2i = \Sigma\ (tik^2 / \lambda k)$ *for* $k = 1...A$ *retained components*
>
> $Qi = \Sigma\ (xij - \hat{x}ij)^2$ *where* $\hat{x}$ *is the model's reconstruction of sample* i

A sample can be atypical by T^2, by Q, or by both at once — each combination points to a different kind of problem.

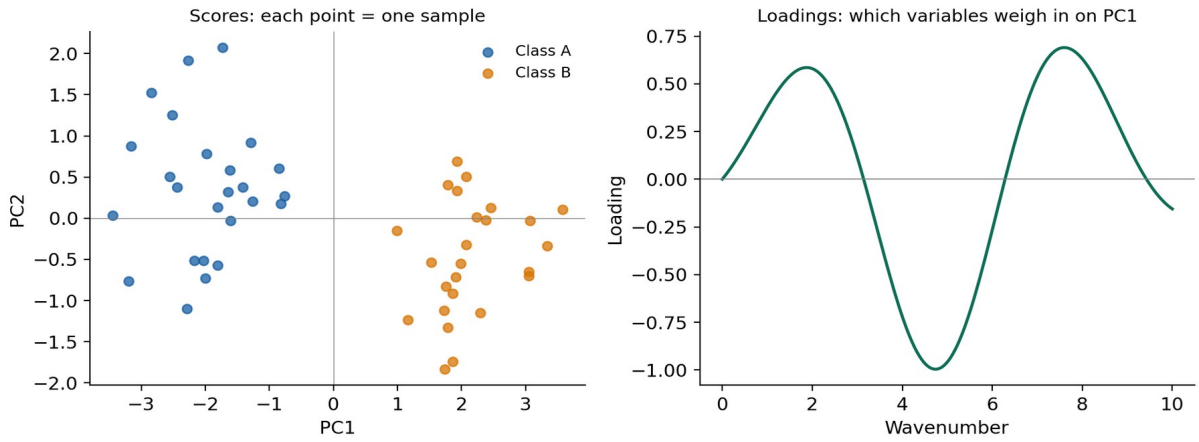


Practical note

The theoretical Q limit (Jackson-Mudholkar approximation) can become unreliable when the retained components already explain nearly 100% of the variance. In those cases, an empirical limit (e.g. the 95th percentile of the observed Q values) is used instead of the theoretical one.

3.3 HCA — Hierarchical Cluster Analysis

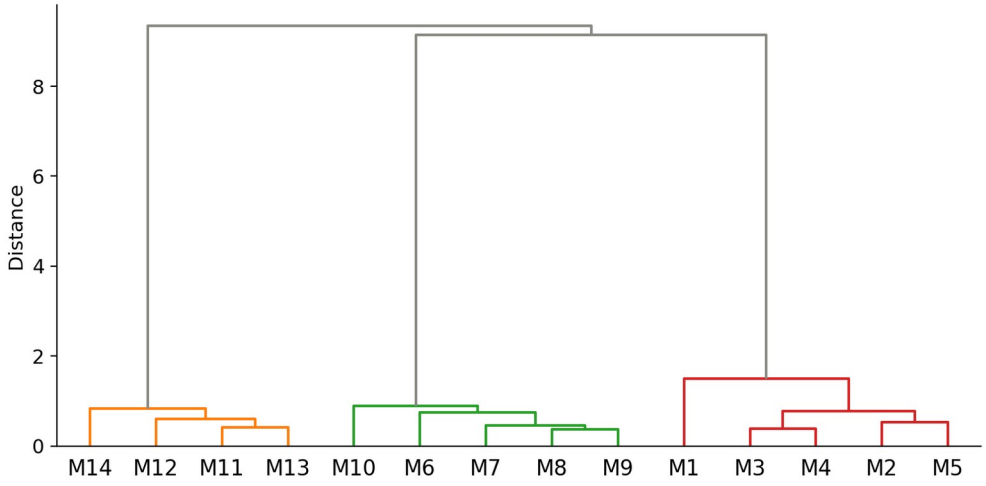
Groups samples hierarchically by similarity (Euclidean or other distance), without needing to decide beforehand how many groups exist. The result is visualized as a dendrogram: the lower the point where two samples merge, the more similar they are to each other.



Where do you cut the dendrogram? A horizontal cut at height h splits the tree into as many clusters as vertical branches it crosses. A long vertical branch without merges means that the groups below it are far apart relative to the rest; cutting inside a long gap gives the most natural number of clusters. In the app you can set the cut

height or the number of clusters and read the resulting purity against your known classes: purity close to 100 % means that the clusters reproduce the classes; low purity means that the variability that dominates your spectra is not the one you are interested in (look at batches, dates, instruments, sample preparation).

The linkage criterion changes the shape of the tree: Ward joins clusters that increase the within-cluster variance the least (compact, similar-sized groups; usually the first choice), average and complete linkage are more tolerant to different shapes, and single linkage tends to chain samples together. Always compare with PCA: if the clusters do not appear in the scores plot either, treat them with suspicion.



3.4 t-SNE and UMAP

PCA is a linear technique. When classes don't separate well in the space of the first principal components, non-linear techniques such as t-SNE or UMAP sometimes reveal complex groupings better. They don't replace PCA (they're harder to interpret quantitatively, and don't give interpretable loadings), but they are a good exploratory complement.

3.5 MCR-ALS (Multivariate Curve Resolution — Alternating Least Squares)

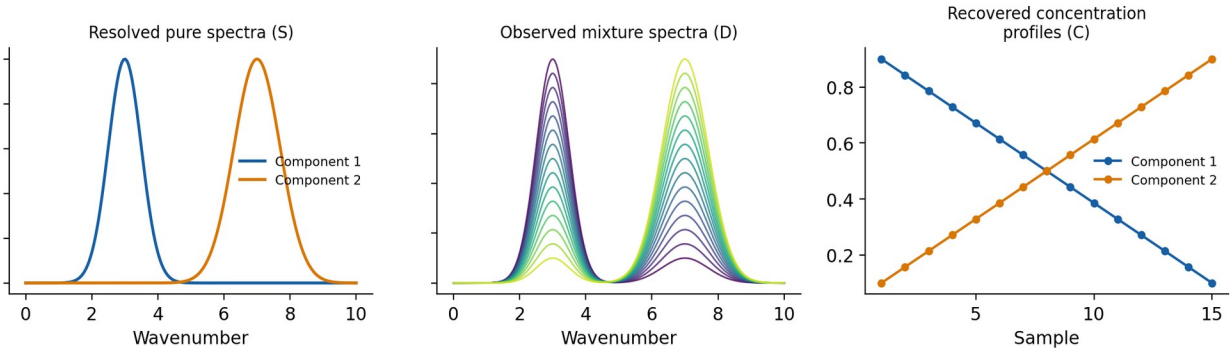
When your samples are MIXTURES, MCR-ALS decomposes the data matrix D into a set of "pure component" spectra (S) and their concentration profile across samples (C), with no needing to know the pure spectra beforehand.

> $D \approx C \cdot S + E \qquad (samples\ x\ wavelengths \approx concentrations\ x\ pure\ spectra\ +\ noise)$

It is solved iteratively: starting from an initial guess (typically from an SVD of D), the algorithm alternates between (1) fixing C and solving for S by least squares, and (2) fixing S and solving for C by least squares — usually under a non-negativity constraint, since real concentrations and spectra can't be negative. This repeats until the reconstruction stops improving.

> $Lack\ of\ fit\ (\%) = 100 \cdot sqrt(\ \Sigma(D - C{\cdot}S)^2 \ /\ \Sigma D^2\ )$

Lack of fit measures how much of the data's variance is NOT explained by the resolved components — lower is better; under roughly 5% is generally considered a good fit for clean data. Choosing too few components leaves real mixture behavior unexplained; too many risks splitting real signal into noise-fitting "components" with no chemical meaning.



From left to right: the true pure spectra, the mixture spectra actually measured, and the concentration profiles MCR-ALS recovers from them alone.

4. Variable selection

A spectrum can have hundreds or thousands of wavenumbers, but usually only some regions are truly informative for a specific prediction task. Reducing the number of variables the model uses can improve its performance (less irrelevant noise) and makes chemical interpretation easier.

4.1 Boruta

Compares the importance of each real variable against randomized copies of itself ("shadow variables", generated by shuffling each variable's values across samples at random) using a Random Forest model. A variable is considered relevant only if it consistently outperforms, across many iterations, the best of its random versions. It is a conservative method: it tends to keep everything that contributes real information, even if the contribution is small.

4.2 Genetic Algorithm

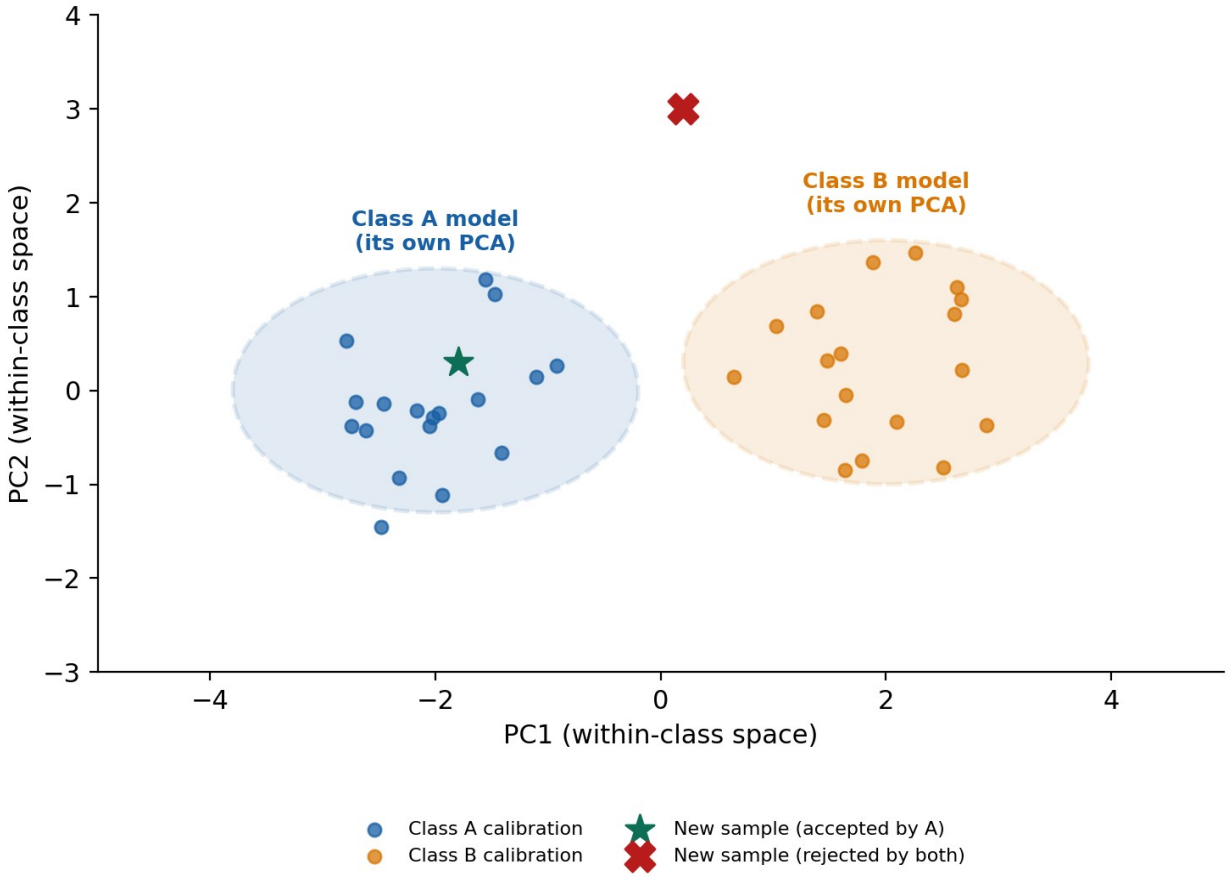
Inspired by biological evolution: each "individual" is a possible subset of variables (encoded as a string of ones and zeros, one per variable). The algorithm evaluates each individual's performance (via cross-validation) and, generation after generation, applies:

- **Selection:** individuals with better performance have a higher chance of "reproducing".
- **Crossover:** the variables chosen by two "parent" individuals are randomly combined to form a "child".
- **Mutation:** with low probability, some variable is switched on or off at random, to avoid getting stuck in a suboptimal solution.
- **Elitism:** the best individual of each generation always survives intact into the next.

After several generations, the best final individual is the proposed variable subset. It is more flexible than Boruta (it can explore specific combinations of variables), but also more computationally expensive and its result depends somewhat more on chance.

5. Class modeling: SIMCA

SIMCA (Soft Independent Modeling of Class Analogies) takes a fundamentally different approach from LDA, PLS-DA, Random Forest, or any other discriminant classifier. A discriminant classifier always forces a pick among the classes it was trained on — even for a sample that doesn't really belong to any of them. SIMCA instead builds one PCA model PER CLASS, and asks, independently for each one, "does this sample look like a member of this class?".



Each class gets its own PCA model (its own ellipse of 'normal' variability). A new sample can be accepted by one class, several, or none.

5.1 How each class model is built

For each class, a PCA is fit using ONLY that class's calibration samples. The number of components is chosen to reach a target cumulative explained variance (e.g. 90%), capped so it never uses more than roughly a third of that class's samples — using too many components relative to sample size leaves almost no residual degrees of freedom, which makes the model fit calibration noise instead of real class structure.

T^2 and Q are then computed exactly as in general outlier detection (section 3.2), but referenced to THAT class's own PCA model — its own components, its own T^2 and Q limits.

5.2 Why the Q limit must be estimated by cross-validation

A subtlety worth understanding: the residual Q computed on the SAME samples used to fit the PCA is systematically optimistic, because PCA is fit specifically to minimize that residual on those samples. Using that in-sample Q directly to set the class boundary makes it too tight — genuinely new samples of the very same class would often get rejected, since their honest residual is larger than what the calibration set showed. The fix is to estimate the Q limit from cross-validated residuals instead: refit the PCA leaving each calibration sample out in turn, project it back in, and use THAT residual (which behaves like a residual on unseen data) to set the boundary. This is what makes a SIMCA class model actually generalize to new samples, rather than only fitting the exact calibration set it was built from.

5.3 Acceptance criterion and per-class metrics

A sample is accepted as a member of a class if it falls within BOTH that class's T^2 limit AND its Q limit. Since every class model is evaluated independently, a sample can end up:

- **Accepted by exactly one class:** a clean call.
- **Accepted by two or more classes:** an ambiguous / overlapping region between classes.
- **Rejected by every class:** a genuine outlier — possibly an unmodeled class, a bad measurement, or something that just doesn't match anything the model has seen.

Two metrics summarize each class model's quality on held-out test samples:

> $$\textit{Sensitivity} = \textit{(true class members correctly accepted)} \,/\, \textit{(all true class members)}$$
>
> $$\textit{Specificity} = \textit{(non-members correctly rejected)} \,/\, \textit{(all non-members)}$$

Interpreting a SIMCA result

High sensitivity AND high specificity for every class means the classes are well separated and each model generalizes well. Low sensitivity for a class means that class's model is too strict (rejecting its own genuine members) — consider more calibration samples or a lower target variance. A large fraction of ambiguous or outlier samples across the whole test set may indicate genuine overlap between classes, or that the classes as defined don't correspond to a clean chemical distinction.

6. Predictive modeling

Once the data is preprocessed and (optionally) reduced to the most relevant variables, a model can be trained to predict a class (classification) or a continuous numeric value (regression) from the spectrum.

6.1 Classification

Algorithm	Core idea
LDA	Finds the linear combination of variables that best separates the classes, maximizing the distance between their centers relative to their internal spread.
PLS-DA	PLS regression on the classes coded as dummy variables; classifies according to the class with the highest predicted value. Widely used in chemometrics because it handles many correlated variables well.
Logistic Regression	Models the probability of belonging to each class via a logistic function over a linear combination of the variables.
Naive Bayes	Applies Bayes' theorem assuming independence between variables; simple and fast, works surprisingly well in many cases.
Random Forest	Combines many decision trees trained on random subsets of data and variables, averaging their votes.
SVM	Finds the boundary (hyperplane) that separates the classes leaving the widest possible margin; with non-linear kernels it can separate curved boundaries.
Decision Tree	Splits the variable space with a sequence of simple if-then rules, easy to interpret.
XGBoost	Builds decision trees sequentially, each one correcting the previous one's errors (gradient boosting); very competitive performance.
KNN	Classifies each sample according to the majority class among its k nearest neighbors in variable space.
Neural Network (MLP)	Layers of connected artificial neurons, able to learn complex non-linear relationships between variables and classes.

6.2 Regression

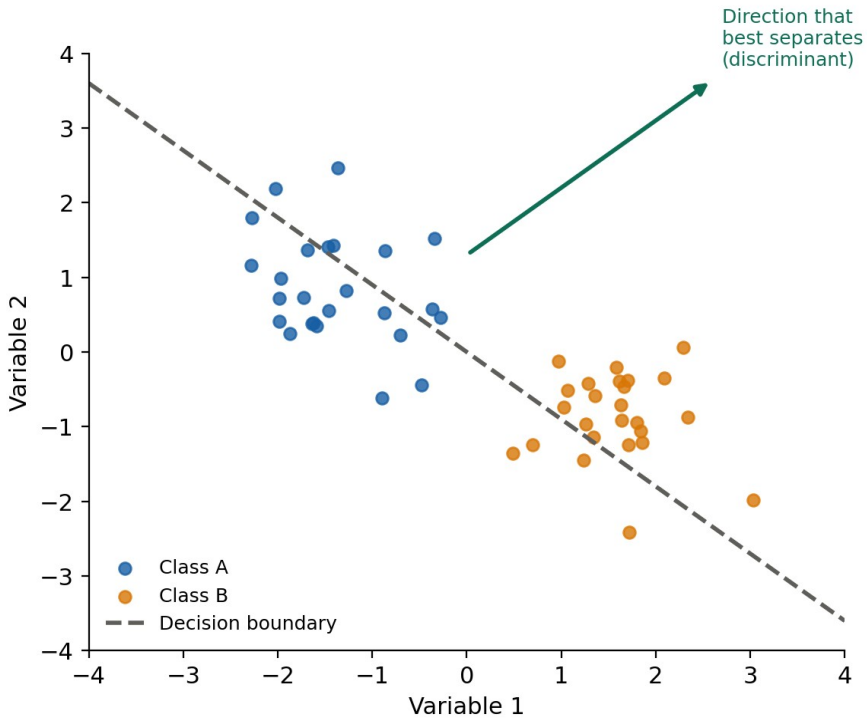
Algorithm	Core idea
PLS	The chemometrics standard: finds components that maximize the covariance between the spectrum and the variable to predict (not just the spectrum's own variance, like PCA).
PCR	Principal Component Regression: PCA first, then a linear regression on the scores.
MLR	Classic multiple linear regression; performs poorly with many correlated variables unless they've already been reduced.
Lasso	Linear regression with a penalty that can push coefficients exactly to zero — performs

Algorithm	Core idea
	variable selection automatically.
Ridge	Linear regression with a penalty that shrinks (without zeroing) coefficients, useful when many variables are correlated.
Elastic Net	Combines Lasso's and Ridge's penalties.
Random Forest / SVM / Decision Tree / XGBoost / KNN	Same algorithms as in classification, adapted to predict a continuous numeric value instead of a class.
Neural Network (MLP)	Same as in classification, adapted for continuous numeric output.

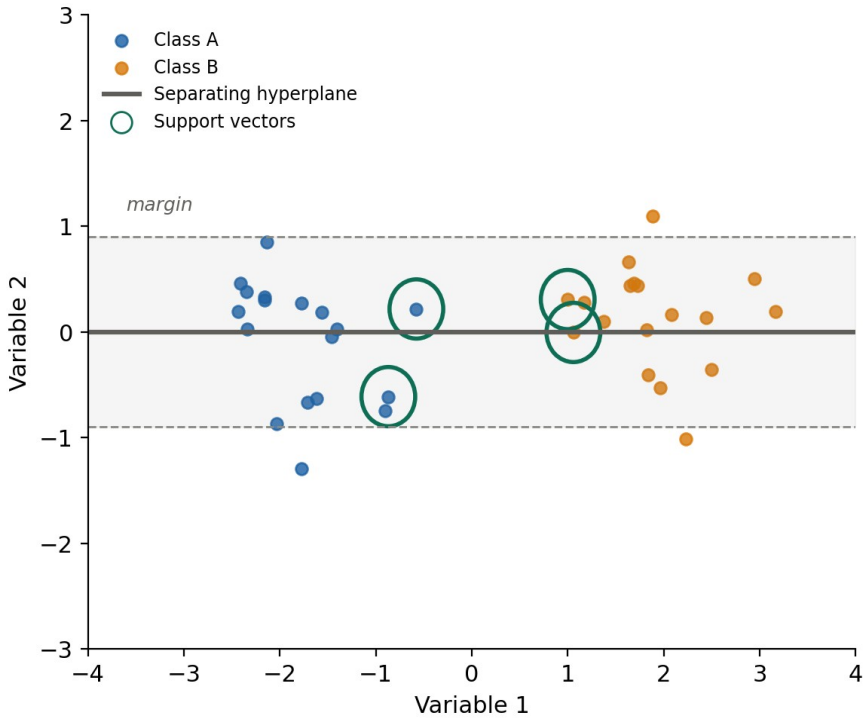
6.3 Illustrative diagrams for selected algorithms

Not all 21 algorithms are illustrated (several share the same visual logic between classification and regression), but these diagrams help build intuition for each family.

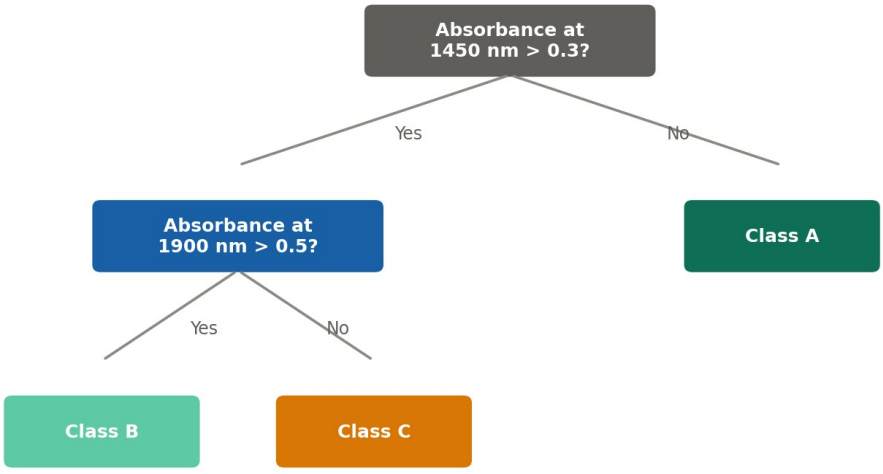
LDA — linear separation



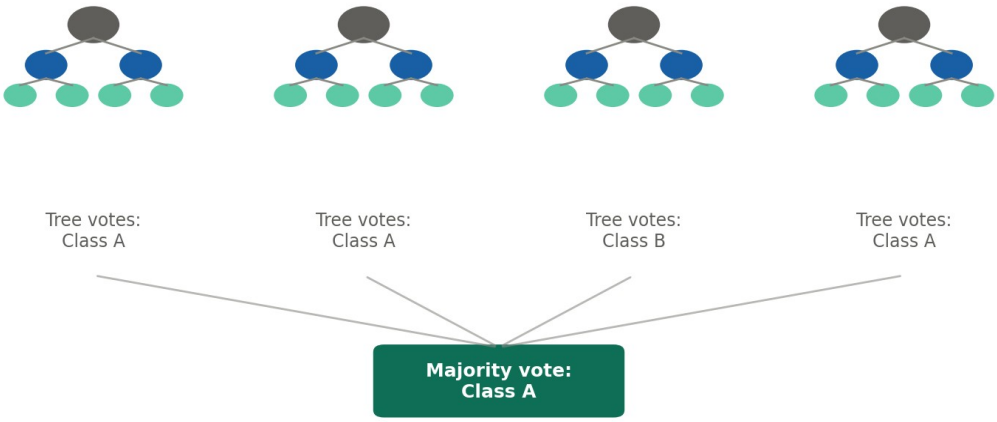
SVM — maximum margin



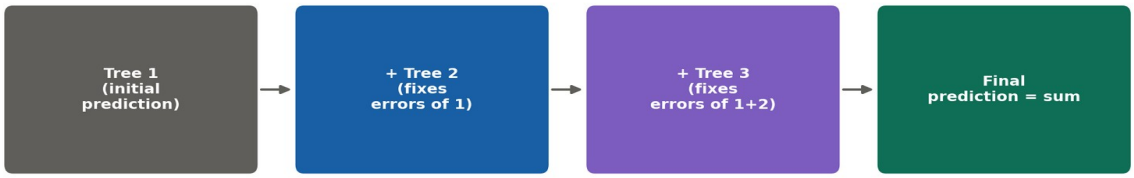
Decision Tree



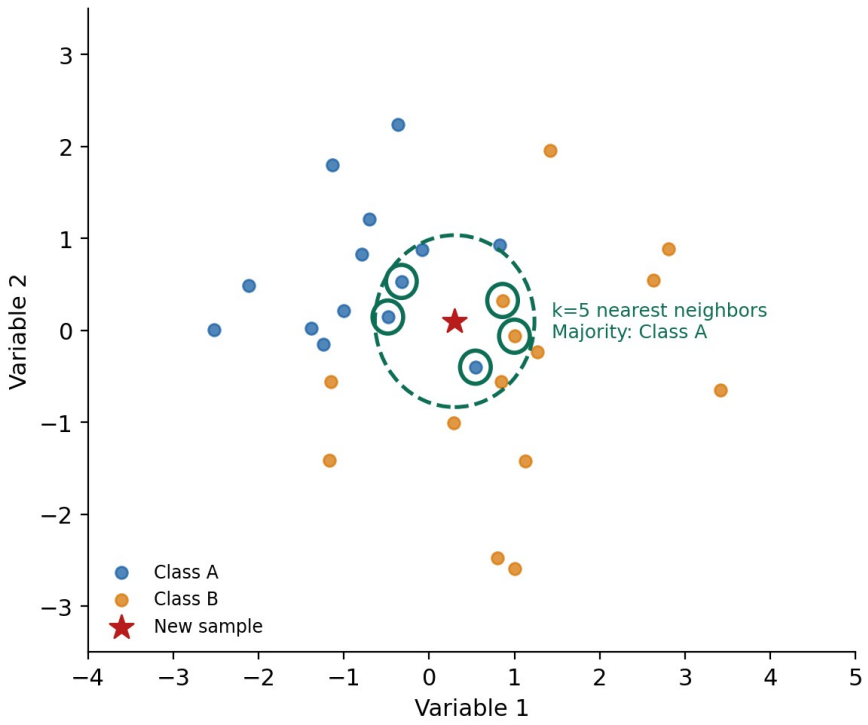
Random Forest — voting across many trees



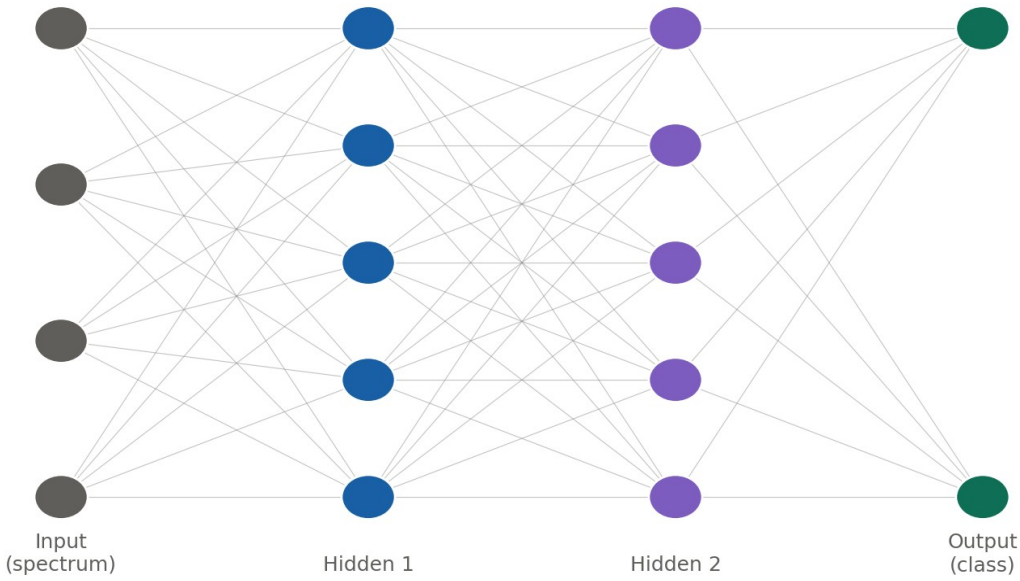
XGBoost — sequential boosting



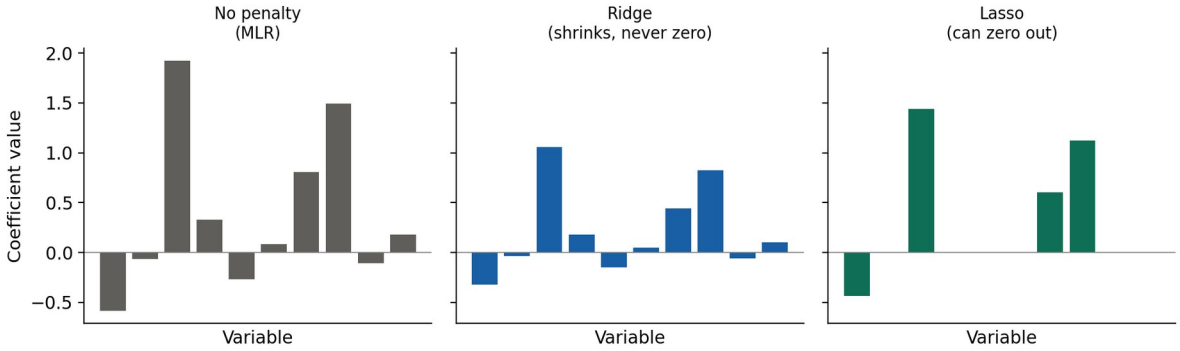
KNN — nearest neighbors



Neural Network (MLP)

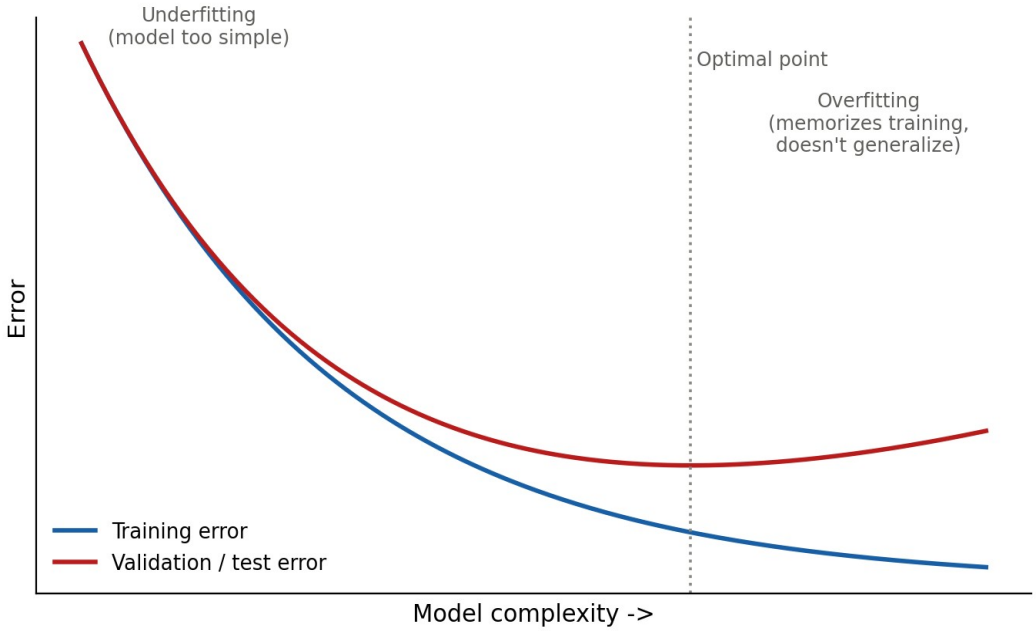


Lasso, Ridge and Elastic Net — regularization



7. Train/test split and cross-validation

A model will always fit the data it was trained on better than it fits new data — the relevant question is how much of that good fit is real learning versus "memorizing" quirks of the training set (overfitting). That is why a model is never evaluated with the same data it was trained on.



7.1 Independent test set

Before training anything, a portion of the samples (e.g. 20%) is set aside and never seen during development. It serves as one final, honest estimate of how the model would perform on truly new samples.

Random split vs. Kennard-Stone

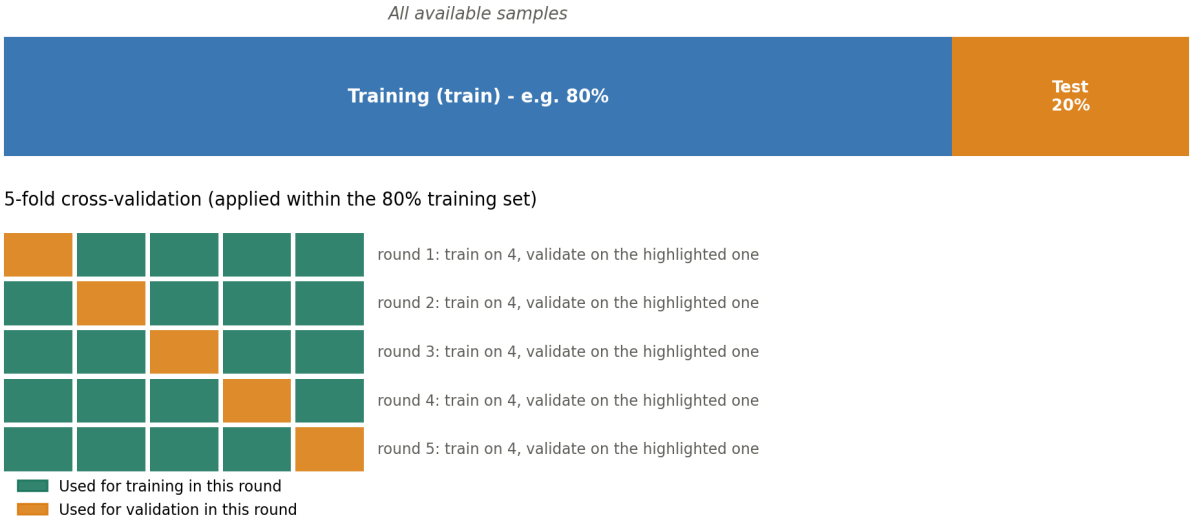
The default way to choose the test set is a random (stratified) split. An alternative, common in chemometrics, is the Kennard-Stone algorithm: it iteratively selects the sample farthest (in Euclidean distance) from the ones already selected, so the chosen TRAINING set broadly covers the X-space, including the more "extreme" samples. The remaining, more "typical" samples become the test set. This tends to give a more stable, representative test estimate than a purely random split, especially with small datasets — though it is not stratified by class, so it's worth checking the resulting class balance.

7.2 Cross-validation (k-fold and Leave-One-Out)

Within the training set, k-fold cross-validation splits the data into k equal parts; over k rounds, the model trains on k-1 parts and validates on the remaining one, rotating which part is left out. In the end, every sample was used exactly once for validation — giving a more stable performance estimate than a single split, without spending extra samples that could be used for training. Typical values: 3, 5, or 10 folds.

Leave-One-Out (LOO) is the extreme case: k equals the number of samples, so each round leaves out and predicts exactly ONE sample, training on all the rest. It is the most thorough option (every sample gets its own

individual held-out prediction) and the standard choice for very small datasets, but it is also the slowest, since it trains one model per sample.

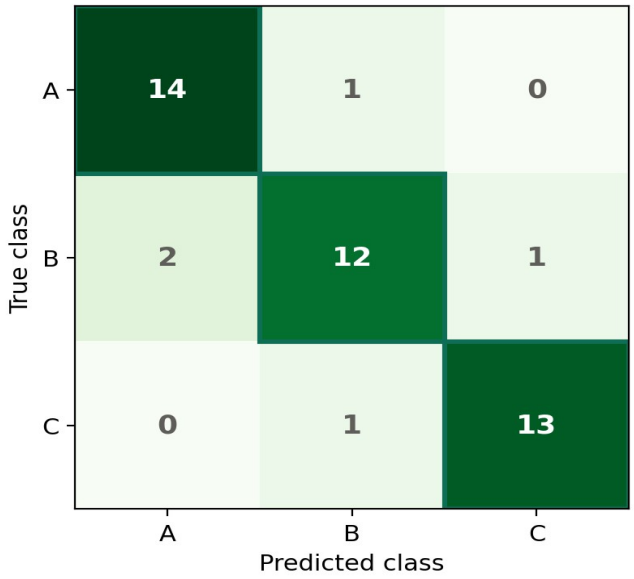


Why not always use a very high k (or LOO)?

With more folds, each model trains on more data and the estimate has less bias, but the process is slower and the estimate can have more variance between folds if there are few samples per class. With fewer folds it's faster, but each model trains on less data. There is no universally correct value — it depends on the size of the dataset.

8. Performance metrics

8.1 Classification



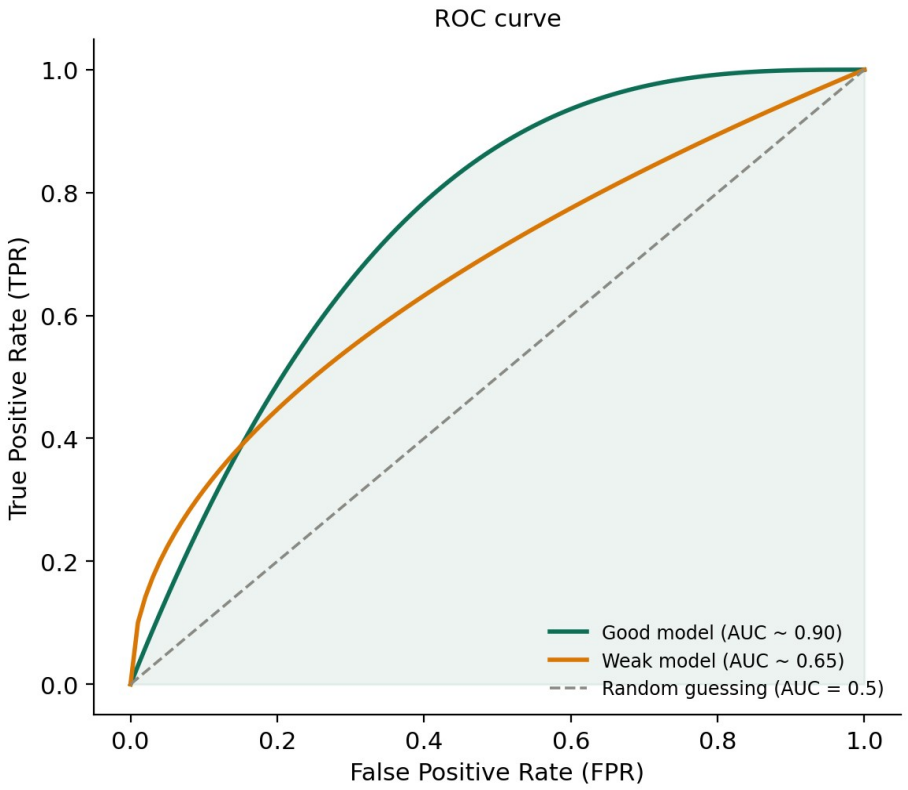
Confusion matrix: the diagonal is correct predictions; off-diagonal, the errors (and which class they were confused with).

Metric	What it measures
Accuracy	Overall proportion of correct predictions. Misleading if classes are imbalanced.
Balanced Accuracy	Average of the per-class accuracy, computed separately for each class — not fooled by classes with many more samples than others.
Sensitivity (Recall)	Of everything that truly IS class X, how much did the model detect? Formula: Sensitivity = TP / (TP + FN).
Specificity	Of everything that truly is NOT class X, how much did the model correctly rule out? Formula: Specificity = TN / (TN + FP).
Precision	Of everything the model labeled as class X, how much really was. Formula: Precision = TP / (TP + FP).
F1 (macro)	Average, across classes, of the harmonic mean between precision and sensitivity: F1 = 2 · Precision · Recall / (Precision + Recall).
Cohen's Kappa	How far above chance the model's agreement with the truth is (0 = like flipping a coin; 1 = perfect). Accounts for the accuracy the model would get purely by chance given the class proportions.
MCC (Matthews Correlation Coefficient)	A single balanced measure that uses all four confusion-matrix quadrants at once, robust to class imbalance. Ranges from -1 (total disagreement) to +1 (perfect); 0 means no better than random.
AUC (Area Under the ROC Curve)	Summarizes how well the model ranks samples by predicted probability of belonging to a class, independent of any single decision threshold. 1.0 = perfect ranking; 0.5 = random.

$$MCC = (TP \cdot TN - FP \cdot FN) / \sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}$$

ROC curve and AUC

The ROC (Receiver Operating Characteristic) curve plots the True Positive Rate (sensitivity) against the False Positive Rate (1 - specificity) as the classification threshold is swept across every possible value. A model that ranks samples well traces a curve that rises quickly toward the top-left corner; a model no better than random guessing traces the diagonal line. The Area Under the Curve (AUC) condenses the whole curve into a single number: the probability that the model ranks a randomly chosen positive sample above a randomly chosen negative one.



A better model (green) sits closer to the top-left corner and has a larger area under its curve than a weaker one (orange). The diagonal represents random guessing.

8.2 Regression

Metric	What it measures
RMSE	Root mean squared error — the model's typical prediction error, in the same units as the Y variable.
MAE	Mean absolute error — similar to RMSE, but less sensitive to occasional large errors.
R^2	Proportion of Y's variability explained by the model (1 = perfect fit; 0 = same as always predicting the average).
RPD	Standard deviation of Y divided by RMSE. Common NIR rule of thumb: RPD > 2 acceptable, > 3 good.

Metric	What it measures
RPIQ	Robust version of RPD: uses Y's interquartile range instead of its standard deviation (less sensitive to outliers in Y).

9. Model diagnostics

Beyond the headline metrics, a few extra diagnostic tools help judge whether a model is trustworthy, whether it would benefit from more data, and whether an apparent improvement over another model is real or just noise.

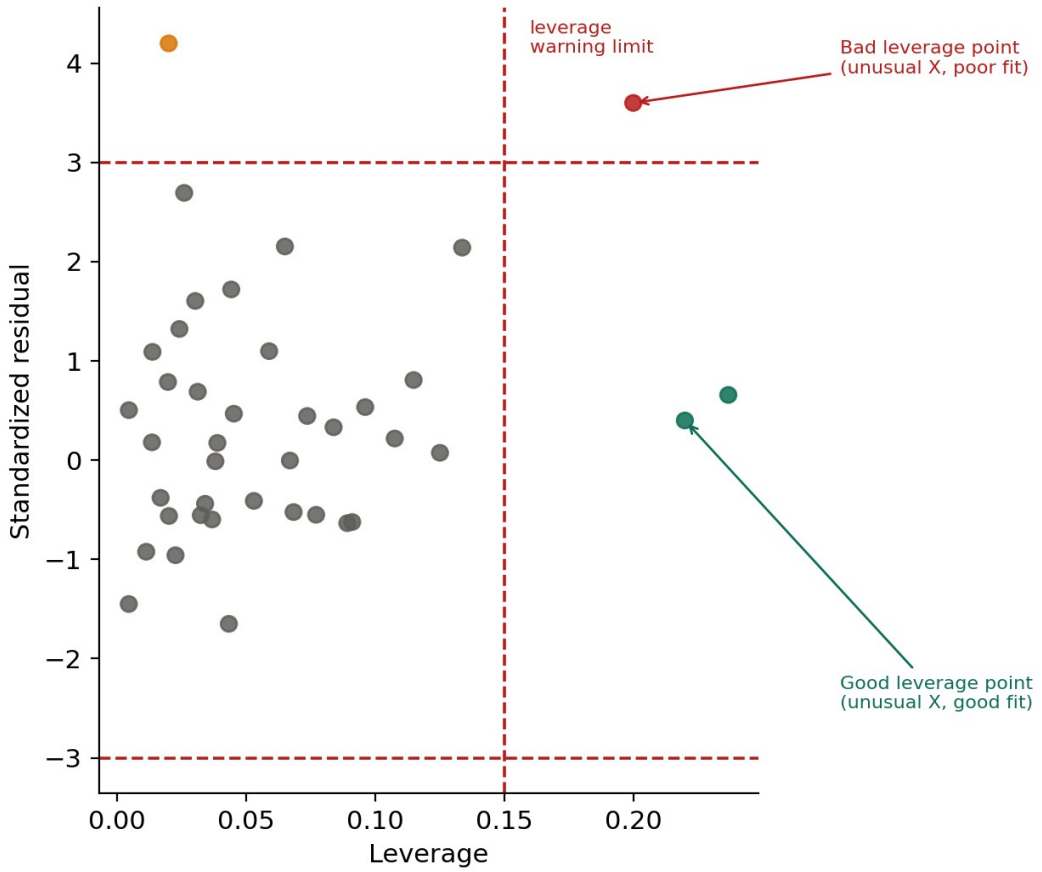
9.1 Leverage / Williams plot

A classic applicability-domain diagnostic, very common in QSAR and chemometrics. Leverage measures how far a sample sits from the center of the spectral space — a generic, algorithm-independent measure computed from a PCA fit on X, so it can be paired with the prediction residual of ANY regression model.

> $hi = 1/n + \Sigma a\ (tia^2 / \Sigma j\ tja^2)$ *(computed from PCA scores t of X)*
>
> $Standardized\ residual = (yi - \hat{y}i) / std(y - \hat{y})$

Plotting leverage (X-axis) against the standardized residual (Y-axis) — the Williams plot — separates samples into four regions:

- **Normal samples:** low leverage, small residual — well within the model's comfort zone.
- **Good leverage points:** high leverage but small residual — an unusual spectrum, but the model still predicts it well. These extend the calibration range usefully.
- **High residual, normal leverage:** a typical-looking spectrum that the model still predicts poorly — worth checking the reference value for that sample.
- **Bad leverage points:** high leverage AND a large residual — an unusual spectrum that the model also predicts poorly. These are the samples most likely to be distorting the model and deserve a closer look (measurement error? a genuinely different sample type?).

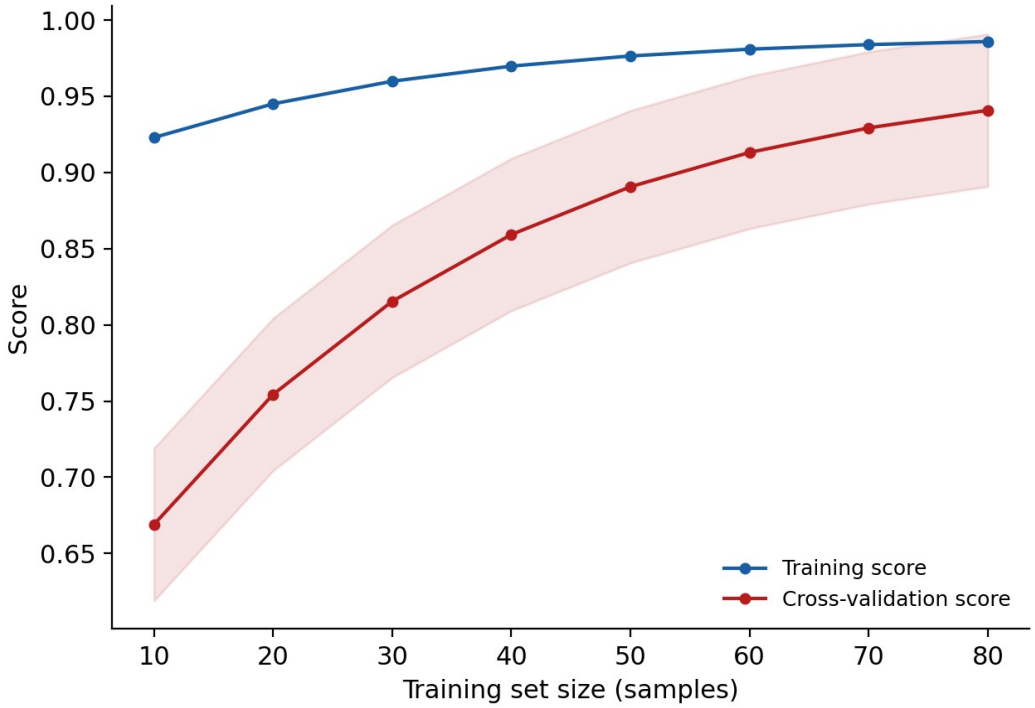


> $h^* = 3 \cdot (A + 1) / n$ *(standard leverage warning threshold; A = components, n = samples)*

9.2 Learning curves

A learning curve plots training and cross-validation performance as a function of how many samples were used to train the model. It helps diagnose whether the model would benefit from more data:

- **Both curves still rising and far apart:** more samples would likely help — the model hasn't seen enough data yet.
- **Curves flat and close together:** the model has plateaued; more samples of the same kind probably won't improve it much. A different algorithm, more variables, or better preprocessing may be needed instead.

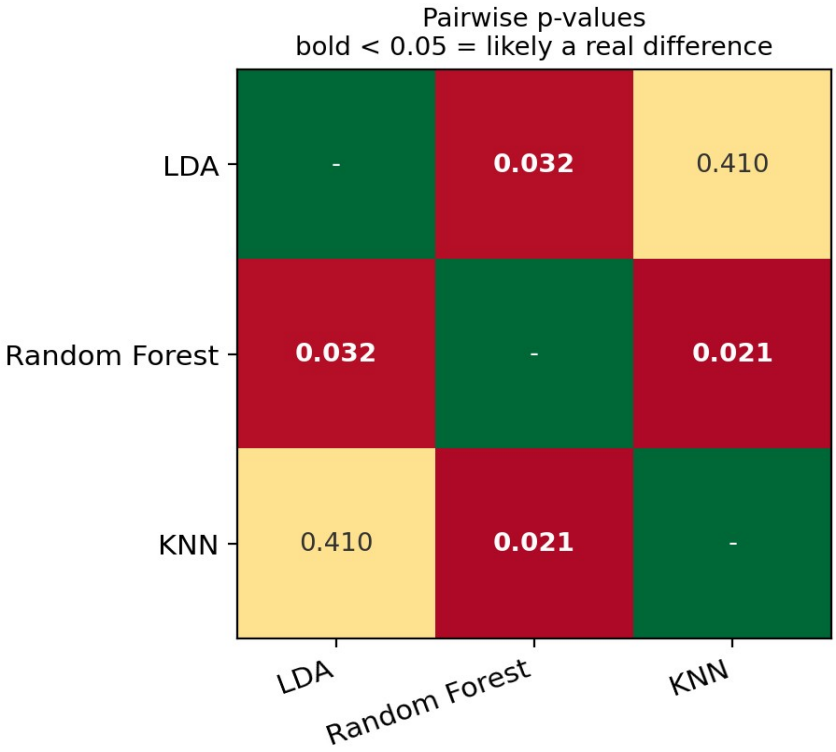


9.3 Statistical comparison between models

A model comparison table often shows one model scoring numerically higher than another — but is that difference real, or could it just be noise from this particular split into folds or this particular test set? A formal statistical test answers that question directly, and the right test depends on whether an independent test set is available:

- **With a held-out test set (preferred):** for classification, McNemar's test compares two classifiers' predictions on the SAME test set, sample by sample — it looks only at the samples where they disagreed, and checks whether one of them was wrong noticeably more often on those. For regression, a paired t-test on the per-sample squared errors of the two models serves the same purpose. Both can genuinely detect overfitting, since a model that overfits will show larger test errors even if its cross-validation score looked fine.
- **Without a test set (cross-validation only):** a paired t-test is run on the per-fold cross-validation scores of the two models (using the exact same folds for both, for a fair comparison). This cannot detect overfitting the way a true held-out test can, since cross-validation folds still influence which samples contributed to the final model choice.

A p-value below 0.05 is the usual convention for calling a difference statistically significant — meaning it's unlikely to be due to chance alone, given this data.



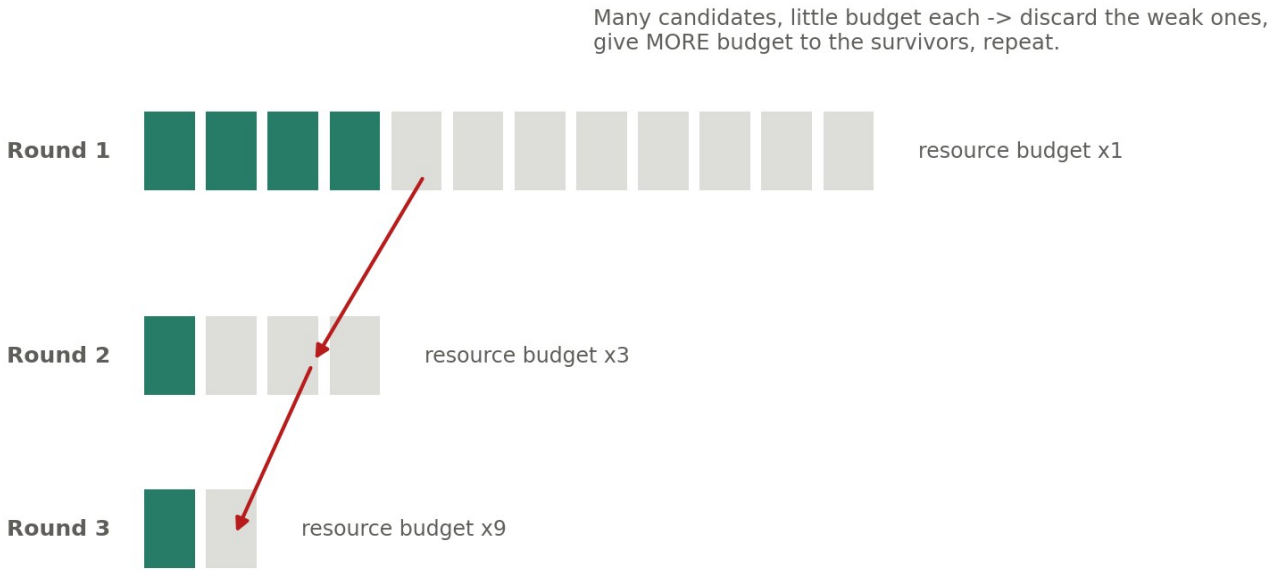
Example output: Random Forest and KNN differ significantly (p=0.021, bold); LDA is not significantly different from either.

10. Hyperparameter optimization

Many algorithms have settings of their own that aren't learned from the data automatically (for example, how many trees to use in a Random Forest, or how many latent variables to keep in a PLS model). Choosing these "hyperparameters" well can meaningfully improve a model's performance.

10.1 Search strategies

- **Grid search:** exhaustively tries every combination of a list of candidate values for each hyperparameter, evaluating each combination by cross-validation. Guarantees finding the best combination within the grid, but can be slow with many hyperparameters or candidate values.
- **Random search:** instead of trying every combination, tries a fixed number of randomly chosen combinations. Usually finds a combination almost as good as the full grid, in much less time.
- **Successive halving:** starts by giving EVERY candidate combination only a small amount of resources (few training samples, or few trees) — enough for a rough estimate. It then discards the weaker half (or more) and gives the survivors a bigger resource budget, repeating until only the best combinations remain, now evaluated with the full budget. This avoids ever spending full computation on combinations that were clearly going to perform badly.



Successive halving: many candidates start with a small budget; only the promising ones survive to receive a larger budget in the next round.

10.2 The app's speed/quality presets

Rather than choosing a raw strategy directly, the app offers three presets that map onto the strategies above:

Preset	What it runs	Best for
Fast	Random search, 8 combinations tried.	A quick first pass, or when you're iterating on

Preset	What it runs	Best for
		preprocessing choices and don't need the best possible model yet.
Balanced (default)	Successive halving from random starting combinations.	The best speed/quality trade-off for most day-to-day use.
Thorough	Successive halving over the FULL grid.	Still checks every combination in the grid, just far more efficiently than classic grid search — use when you want the most complete search you can afford.

For Random Forest and XGBoost, successive halving grows the NUMBER OF TREES across rounds (since that is the natural "resource" for those models); for every other algorithm, it grows the number of TRAINING SAMPLES used instead. With very few samples, or with Leave-One-Out cross-validation combined with a sample-based resource, the app automatically falls back to a plain grid/random search, since successive halving's resource-scheduling assumptions don't hold in those edge cases.

> **Important**
>
> Hyperparameter optimization must always be done using cross-validation on the training set, never by looking at the result on the test set — otherwise the test set stops being an honest estimate of performance on new data.

11. How far can a prediction be trusted?

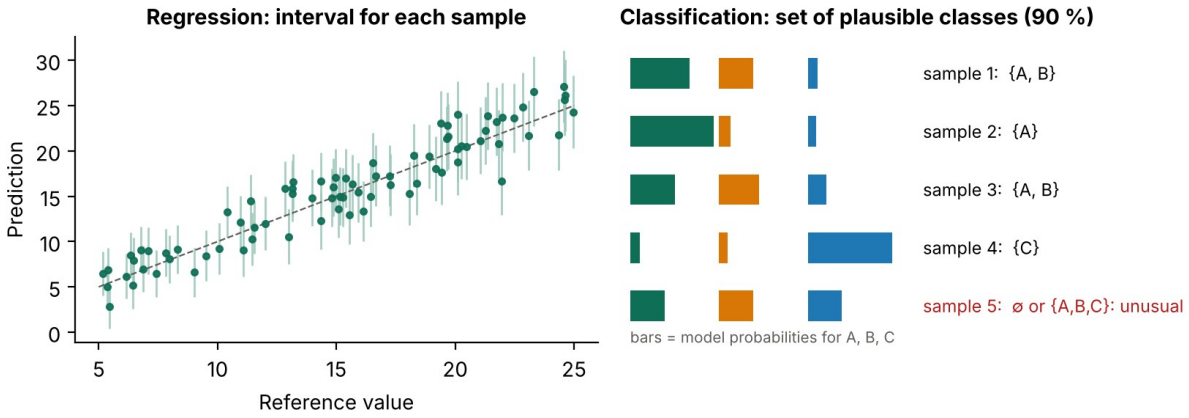
A model always returns a number or a class, even for a sample that has nothing to do with the training set. This chapter covers the tools that qualify each prediction: an interval or set with guaranteed coverage, a check that the sample resembles the training data, an explanation of what drove the prediction, and a global view of which spectral regions the model uses. In the app they are in Modeling & prediction → Prediction ("Reliability tools"), SIMCA and Model importance. None of them runs until you tick its box or click its button.

11.1 Conformal prediction

The usual rule of thumb ŷ ± 1.96·RMSE assumes that errors are normal and equally large everywhere; neither is usually true. Conformal prediction replaces the assumption by evidence: the out-of-fold residuals of the cross-validation, which are honest errors made on samples the model had not seen, are used to calibrate the interval.

> $interval = [\,\hat{y} - q\,,\ \hat{y} + q\,] \qquad q = the\ \lceil (n+1)(1-\alpha) \rceil / n\ quantile\ of\ |y_i - \hat{y}_i|\ (out\text{-}of\text{-}fold)$

With α = 0.10 the interval contains the true value in about 90 % of future samples, whatever the algorithm. In classification the same idea gives a set of classes instead of a single label: the classes whose probability is high enough, with the threshold calibrated separately for each class so that rare classes keep their coverage. A set with one class is a confident answer; a set with two or more is an honest "it could be A or B at 90 % confidence"; an empty set means that no class fits and the sample is unusual (a new class, an adulterant, a problem with the measurement).



What the guarantee assumes

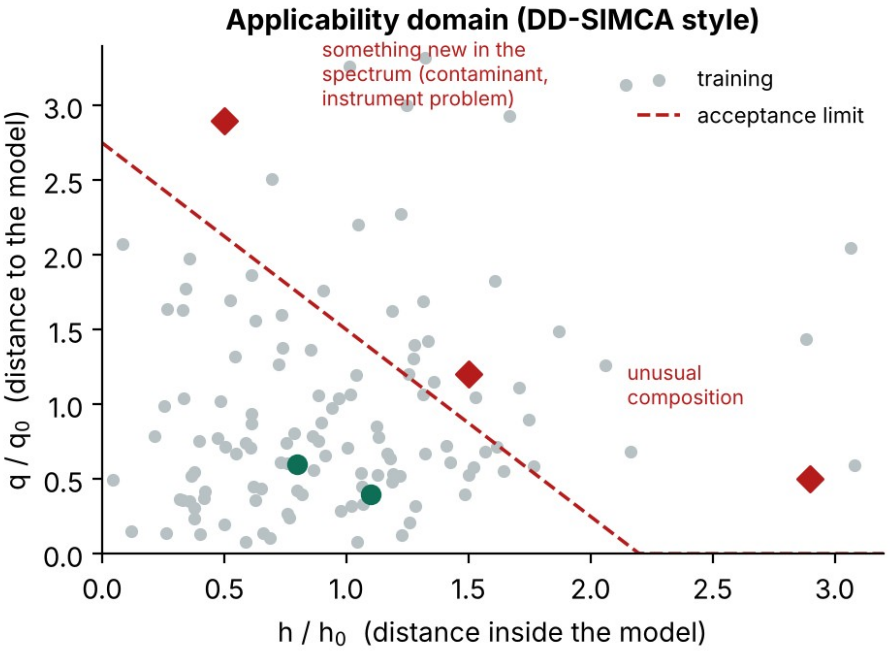
The coverage is guaranteed on average if new samples behave like the calibration samples (technically, they are exchangeable with them). A new instrument, a new matrix or a drifting method breaks this assumption without any warning from the conformal interval itself — which is why it should be read together with the applicability-domain check below.

11.2 Applicability domain and data-driven SIMCA

A model can only be trusted for samples that resemble those it was built on. The domain check describes the training data with a PCA model and measures, for every new sample, two distances: h, how far its scores are from the centre inside the model, and q, how much of the spectrum the model cannot reproduce. Both are divided by their typical value in the training set (h_0 and q_0) so that they have comparable scale.

$$c = N_h \cdot (h / h_0) + N_q \cdot (q / q_0) \sim \chi^2 \text{ with } N_h + N_q \text{ degrees of freedom} \qquad N = 2 \cdot (mean / sd)^2$$

The degrees of freedom are estimated from the mean and the spread of the training distances (the method of moments), so no assumption on the shape of the data is needed beyond this approximation. A sample is accepted when c is below the χ^2 quantile for the chosen α; samples above a second, stricter quantile are flagged as extreme. In the acceptance plot, points to the right of the limit have an unusual composition inside the model's space; points above it contain something the model cannot describe — a contaminant, a problem with the instrument.

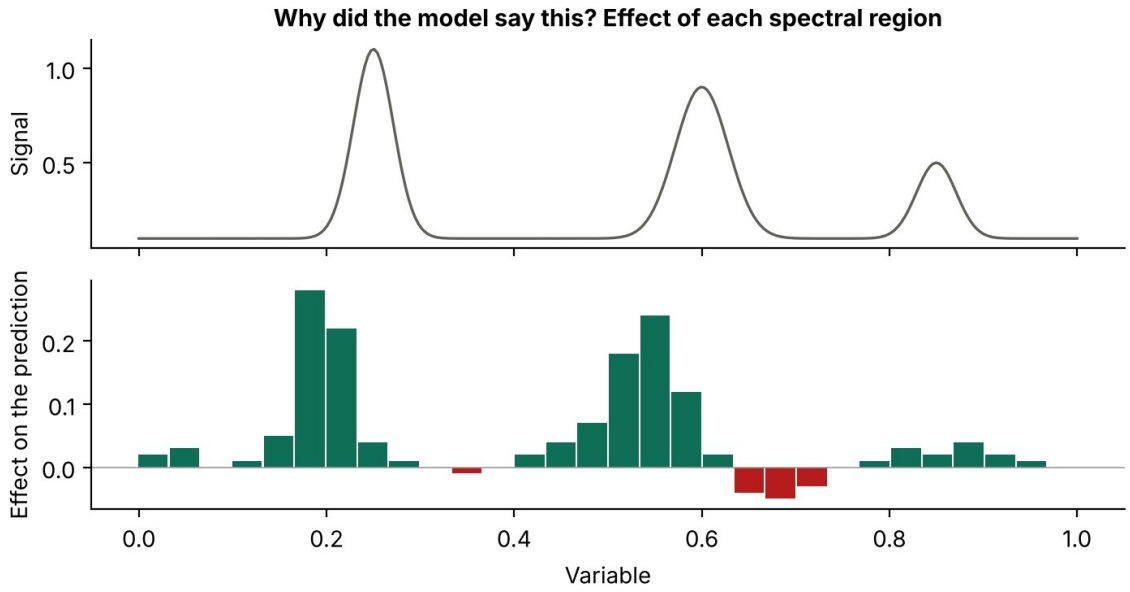


SIMCA can use this rule too. In the SIMCA tab the choice is between the classical rule (separate limits for T^2 and Q, a rectangle) and the data-driven rule (the combined distance, a triangular region) of DD-SIMCA, which usually reproduces the intended false-rejection rate more faithfully when the training set is small.

> **The "explained variance" setting of a class model**
>
> The explained variance chosen for each class sets how many components describe the class. Higher explained variance = more components = a tighter, more demanding class model: foreign or adulterated samples are detected more easily, but genuine samples of the class are also rejected more often (lower sensitivity). Lower explained variance gives a looser model: fewer genuine samples are rejected, but adulterants are more likely to pass. Choose it by looking at both the sensitivity of the class and the specificity against the other classes, using samples not used to build the model.

11.3 Why did the model say that? Explaining one prediction

To explain a single prediction the app cuts the spectrum into windows and, one window at a time, replaces it with the training average; the change in the prediction (or in the probability of a chosen class) is the effect of that window. Green bars push the prediction up, red bars push it down. The most influential regions can be assigned to functional groups (chapter 12).



What this is — and is not

It is a local, model-based explanation: it tells what the model used for this sample, not what is chemically responsible. When neighbouring regions are correlated, the effect is shared among them. A sensible explanation (bands that a chemist would expect) is reassuring; an explanation based on baseline or noise regions is a warning that the model is not relying on chemistry.

12. From bands to chemistry, and tabular data

12.1 Functional-group assignment

Wherever the app highlights spectral regions — variables selected by Boruta or the genetic algorithm, the PCA loadings, the PC finder, the explanation of a prediction — it can list the functional groups, overtones and combination bands, or proton types that usually absorb (or resonate) there, for FT-MIR, NIR (nm or cm^{-1}), Raman and ^{1}H NMR. Next to each group the structure is drawn: the bond that vibrates, or the protons that resonate, in red, and the rest of the molecule in black.

A few typical landmarks illustrate the idea (all approximate): in the NIR, the first overtone of O–H appears near 1400–1450 nm, the water combination band near 1940 nm, N–H first overtones near 1500–1530 nm, C–H first overtones near 1650–1800 nm and C–H combination bands between about 2270 and 2350 nm; in the mid-infrared, O–H and N–H stretches lie at 3200–3600 cm^{-1}, C–H stretches at 2800–3000 cm^{-1} and C=O stretches at 1650–1800 cm^{-1}; in Raman, aromatic ring modes appear near 1000 and 1600 cm^{-1} and C≡N near 2200–2260 cm^{-1}; in ^{1}H NMR, aliphatic protons resonate at about 0.5–2 ppm, protons next to a carbonyl or double bond at 2–3 ppm, protons on carbons bearing oxygen at 3.3–4.5 ppm, olefinic and anomeric protons at 4.5–6.5 ppm, aromatic protons at 6.5–8.5 ppm and aldehydes at 9–10 ppm.

Indicative values only

The ranges in the app and in this guide are typical values. Real positions shift with the matrix, solvent, pH, temperature, hydrogen bonding and neighbouring groups, and several groups overlap in the same region. Always confirm an assignment against the literature or a reference spectrum before drawing conclusions.

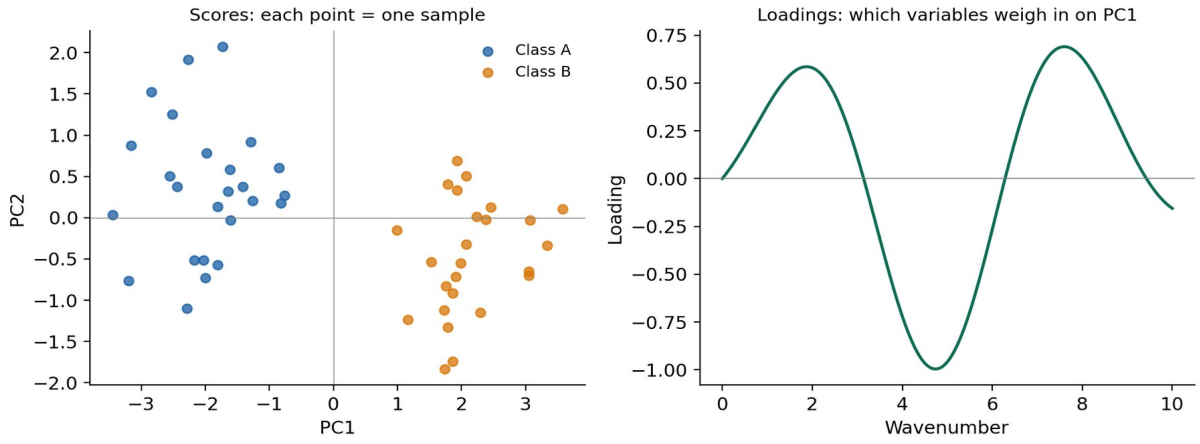
12.2 Tabular data and metabolomics mode

Not every dataset is a spectrum. In the tabular mode each column is a named variable (a metabolite, a clinical analysis, a compound concentration). The names are kept in every table and plot while the technical axis is only the position 1…p. Pre-treatments suited to tables are available (imputation of missing values, removal of constant variables, log transformation, centring, autoscaling and Pareto scaling), the pre-treatment is stored with each saved model, and new samples are matched to the model by variable name, so the order of the columns does not matter. Functional-group assignment is not offered for tabular variables.

13. Your learning path in the app

Academy suggests a path of nine steps and shows how far you have explored it (the progress bar on the Home page and in the sidebar). You do not need to follow it strictly, but each step answers a question that the next one assumes.

- **1 · Load data:** Load spectra or a table, set IDs, classes and/or reference values.
- **2 · Preprocess:** Remove baseline / scatter / scale differences (or transform + scale a table).
- **3 · Explore (PCA):** Look at the structure of the data: groups, trends, anomalies.
- **4 · Check outliers:** Find samples that do not behave like the others.
- **5 · Cluster:** Do samples group the way you expect? (optional but very instructive)
- **6 · Screen models:** Compare many preprocessing × algorithm combinations fairly.
- **7 · Refine a model:** Classification / Regression / SIMCA with tuning and validation.
- **8 · Predict new samples:** Use a saved model, with its domain of applicability.
- **9 · Report:** Export a traceable report of everything you did.

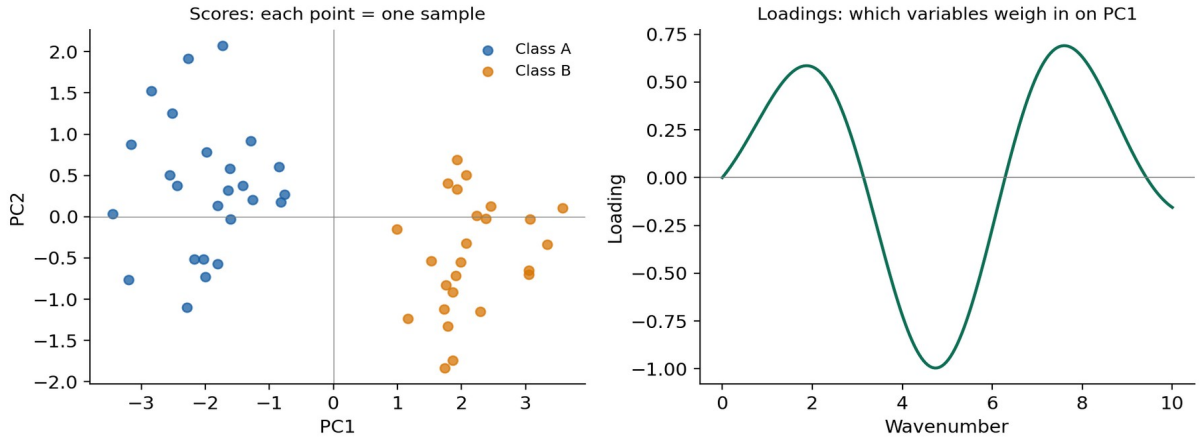


13.1 The three guide buttons

- **What's next?:** a short, personalised recommendation based on what you have already done (for example: after PCA, check outliers before modeling).
- **How to interpret?:** explains every metric or plot of the current result, how to read it, and a rule of thumb with a traffic-light colour for the values you obtained.
- **Pros & cons:** strengths, weaknesses and typical use of the algorithm you selected (PLS-DA, SVM, Random Forest…), so you can choose a model for a reason.

13.2 Observation boxes and the final report

Under the main plots you will find an observation box. Write what you see (for example, "PC2 separates the two harvests"). Every observation is saved with the figure it belongs to and is printed in the final report together with the settings and the traceability of the session, so the report tells the story of your analysis, not only the numbers.



14. Metric cheat sheet

The same explanations appear next to your results under the How to interpret? button. This is a compact reference to keep at hand.

14.1 Classification

- **Accuracy:** Fraction of samples classified correctly. How to read it: Easy to read but misleading when classes are unbalanced (90 % of samples in one class → a model that always answers that class gets 90 %). Rule of thumb: Use it only with balanced classes.
- **Balanced accuracy:** Average of the recall of each class (every class weighs the same). How to read it: The honest version of accuracy for unbalanced data. 1 = perfect, 1/k = random guessing (k classes). Rule of thumb: ≥ 0.9 good · 0.7–0.9 acceptable · < 0.7 weak.
- **Sensitivity / recall (per class):** Of the samples that REALLY belong to a class, how many did the model find? How to read it: Low sensitivity = samples of that class are being missed (false negatives). Rule of thumb: Critical when missing a case is costly (disease, adulteration).

- **Specificity:** Of the samples that do NOT belong to a class, how many did the model correctly leave out? How to read it: Low specificity = many false alarms (false positives). Rule of thumb: Critical when a false alarm is costly (rejecting good batches).
- **Precision:** Of the samples the model assigned to a class, how many really belong to it? How to read it: Low precision = the class label cannot be trusted when it appears. Rule of thumb: Look at precision and recall together (F1).
- **F1-score:** Harmonic mean of precision and recall (0–1). How to read it: Penalises models that are good on one and bad on the other. Rule of thumb: ≥ 0.9 good.
- **Cohen's kappa:** Agreement between prediction and truth beyond what chance would give (−1 to 1). How to read it: 0 = chance, 1 = perfect. Rule of thumb: > 0.8 almost perfect · 0.6–0.8 substantial · 0.4–0.6 moderate · < 0.4 weak.
- **MCC (Matthews correlation):** A single balanced score that uses all four cells of the confusion matrix (−1 to 1). How to read it: Robust with unbalanced classes; 0 = chance. Rule of thumb: > 0.7 good.
- **AUC (ROC):** Probability that the model ranks a random sample of a class above a random sample of another (0.5–1). How to read it: Measures how well classes are separated regardless of the decision threshold. Rule of thumb: > 0.9 excellent · 0.8–0.9 good · 0.7–0.8 fair · 0.5 = random.
- **Confusion matrix:** Rows = true class, columns = predicted class. How to read it: The diagonal is correct. Off-diagonal cells tell you WHICH classes get confused — often neighbours chemically (e.g. two close origins). Rule of thumb: Use it to decide what to improve: more samples of the confused classes, other preprocessing, or merging classes.
- **Cross-validation (CV) vs. test:** CV is computed inside the training set (rotating which part is left out). The test set is never touched until the end. How to read it: CV ≈ test: the estimate is reliable. CV much better than test (> 10–15 points): overfitting, or the test set differs from the training set. The test result is the one to report. Rule of thumb: With < 30 samples both are noisy: report them with caution and prefer repeated CV.

14.2 Regression

- **RMSE (RMSEC / RMSECV / RMSEP):** Typical size of the prediction error, in the units of the property you predict. How to read it: C = calibration (training, optimistic), CV = cross-validation, P = prediction on the test set. Compare RMSECV and RMSEP: they should be similar. Rule of thumb: What matters is the error compared with the range of y and with the method's required precision.
- **R^2:** Fraction of the variance of y explained by the model (1 = perfect, 0 = as good as predicting the mean). How to read it: R^2 in calibration is always flattering. Look at R^2 of CV and test. Rule of thumb: > 0.9 excellent · 0.8–0.9 good · 0.6–0.8 screening only · < 0.6 weak.
- **RPD:** Standard deviation of y divided by the prediction error (SD/RMSE). How to read it: How many times smaller the error is than the natural spread of the samples. Rule of thumb: > 3 good for quantification · 2–3 screening/rough · < 1.5 not usable. (Rule from NIR agricultural/food literature.)
- **RPIQ:** Interquartile range of y divided by the error — like RPD but robust to skewed distributions. How to read it: Preferred when y is not normally distributed (soils, biological data). Rule of thumb: > 4 good · 2.5–4 acceptable.
- **MAE:** Mean absolute error — like RMSE but without extra weight on large errors. How to read it: If RMSE ≫ MAE a few predictions are very wrong; look at the residual plot. Rule of thumb: Report it together with RMSE.

- **Bias:** Average of (predicted − measured). How to read it: Should be ≈ 0. A bias means the model systematically over/under-predicts (often a sign of a shift between training and new samples). Rule of thumb: Check the slope/intercept of predicted vs. measured too.
- **Predicted vs. measured plot:** Ideal = points on the 1:1 line. How to read it: Curvature = non-linear relationship; fan shape = error grows with concentration; separate clouds = hidden groups. Rule of thumb: Colour by batch/class to find the cause of the pattern.
- **Residuals plot:** Measured − predicted for each sample. How to read it: Should look like random noise around 0. Trends or isolated large residuals reveal model problems or outliers.
- **Latent variables (PLS/PCR):** Number of components used. How to read it: RMSECV usually drops and then rises: choose the smallest number near the minimum (parsimony). Rule of thumb: Too many components fit the noise: good calibration, bad prediction.
- **CV vs. test:** Same logic as in classification. How to read it: RMSEP much larger than RMSECV = overfitting or non-representative split. Rule of thumb: Report RMSEP (test set) as the final figure.

14.3 SIMCA

- **One PCA model per class:** SIMCA does not draw boundaries between classes; each class gets its own PCA model and a sample is accepted if it fits that model (T^2 and Q inside the limits). How to read it: That means a sample can be accepted by ONE class, by SEVERAL (ambiguous) or by NONE (outlier / unknown class). Rule of thumb: Great for authentication ('is this really class A?') and for detecting samples of unknown classes.
- **Sensitivity of a class:** Percentage of the class's own test samples accepted by its model. How to read it: Low = the model is too strict (raise α, use more components). Rule of thumb: Close to $(1 - \alpha) \times 100$ for a good model, e.g. ~95 %.
- **Specificity of a class:** Percentage of samples of OTHER classes rejected by this model. How to read it: Low = the model is too permissive: classes overlap or too many components were kept. Rule of thumb: > 90 % good.
- **Accepted by several / by none:** Ambiguous vs. unrecognised samples. How to read it: Many 'several' → classes are not distinguishable by this representation. Many 'none' → outliers, missing classes or a model that is too strict.
- **Significance level α:** Probability that a good sample of the class is wrongly rejected. How to read it: Lower α → wider limits → higher sensitivity, lower specificity (and vice versa). Rule of thumb: 0.05 is the usual compromise.
- **Classical vs. DD-SIMCA:** Classical SIMCA uses separate T^2 and Q limits. DD-SIMCA combines them in one data-driven acceptance area estimated from the training data. How to read it: DD-SIMCA is the recommended modern formulation and gives a more reliable false-alarm rate.
- **Cooman's plot / T^2–Q plot:** Distance of every sample to two class models at once, or T^2 vs. Q for one class. How to read it: Samples in the corner of both models are ambiguous; those far from both are unknown.

14.4 Outliers (T^2 and Q)

- **Hotelling's T^2:** Distance of a sample from the centre of the data INSIDE the PCA model (how extreme its scores are). How to read it: High T^2 = the sample is unusual in the same directions as the others, but

more extreme (e.g. a very high concentration). Rule of thumb: Above the limit (α = 0.05 by default): extreme but 'explained by the model'. Often still valid.

- **Q residual (SPE):** How much of the sample's spectrum is NOT explained by the PCA model (distance to the model plane). How to read it: High Q = something the model has never seen: contamination, a spike, a different matrix, instrument fault. Rule of thumb: Above the limit: look at the raw spectrum before keeping the sample.
- **Four quadrants:** Low/low = normal. High T^2 only = extreme but consistent. High Q only = new kind of variation. High both = very suspicious. How to read it: Prioritise the top-right quadrant. Rule of thumb: With α = 0.05 about 5 % of good samples are flagged by chance — do not delete them blindly.
- **Before excluding:** Excluding is reversible here, and every exclusion is logged in the report. How to read it: Ask: is there a physical reason (bad measurement, mislabeled, different product)? Then exclude. If not, keep it and mention it. Rule of thumb: Excluding more than ~5–10 % of the samples is a sign the preprocessing or the sampling should be revisited.

14.5 PCA

- **Explained variance (per PC and cumulative):** How much of the total variation in the data each principal component (PC) captures. How to read it: PC1 holds the most variation, PC2 the most of what is left, and so on. The scree plot shows where adding PCs stops paying off (the 'elbow'). Rule of thumb: Keep enough PCs to reach ~90–99 % cumulative variance for spectra (fewer for noisy tables). More PCs than needed just model noise.
- **Scores plot:** Each point is a sample, placed by its coordinates on the PCs. How to read it: Samples that are close have similar spectra/profiles. Groups = classes or batches; a gradient = a trend (e.g. concentration); an isolated point = a candidate outlier. Rule of thumb: Colour by class: if classes separate in PC1–PC2, a classifier will probably work. Overlap does not mean failure — other PCs or a supervised model may still separate them.
- **Loadings:** The weight of each original variable (wavenumber, ppm, compound…) in a PC. How to read it: Large positive/negative loadings mark the variables responsible for the separation seen in the scores. For spectra they look like spectra: peaks point to the chemistry. Rule of thumb: Compare the loading peaks with known bands (Learn tab) to link the model to the chemistry.
- **Scores vs. batches / time:** If a PC separates measurement days, instruments or operators, that PC is capturing a nuisance effect. How to read it: Colour by batch. If batches separate more than classes, improve preprocessing or randomise the measurement order next time. Rule of thumb: Always check for this before trusting a model.

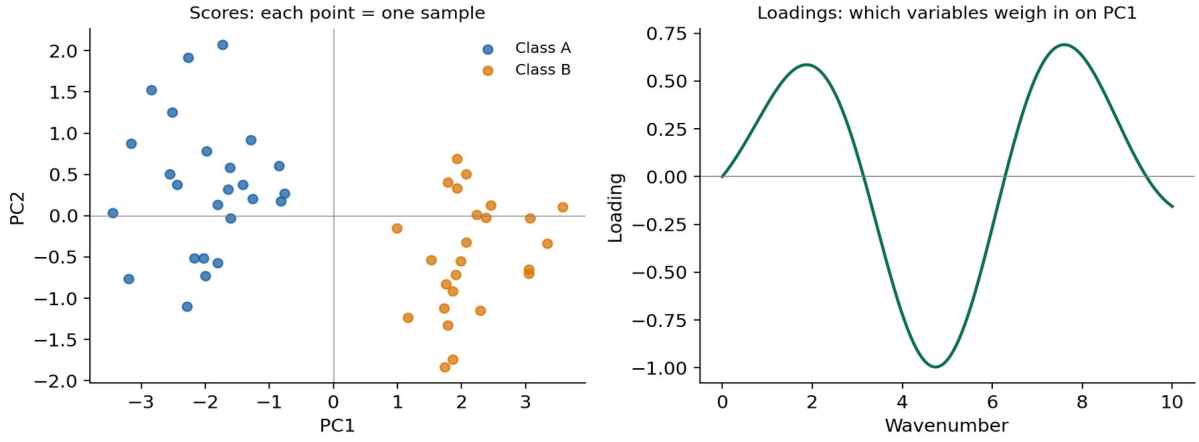
14.6 Dendrogram

- **Branch height:** The distance at which two groups merge. Low merges = very similar samples; tall merges = very different. How to read it: Look for a few tall branches with many short ones below: that is natural group structure. Rule of thumb: Cut where there is a large jump in height to decide the number of clusters.
- **Leaf colours (classes):** Labels coloured by the known class. How to read it: If every cluster is one colour, the classes are naturally separable (pure clusters). Mixed colours = the classes overlap in this representation. Rule of thumb: The app reports a purity: the fraction of samples that fall in the majority class of their cluster (≥ 0.9 excellent, 0.7–0.9 acceptable, < 0.7 poor).

- **Linkage method:** How the distance between two clusters is defined. How to read it: Ward → compact, similar-sized clusters (best default for spectra). Average → compromise. Complete → tight clusters, sensitive to outliers. Single → chains, rarely good for spectra. Rule of thumb: If purity is poor, try another linkage and/or a different preprocessing before concluding anything.

14.7 Prediction

- **Predicted class / value:** What the saved model answers for each new sample, after applying EXACTLY the same preprocessing. How to read it: Always read it together with the confidence or interval and the domain check.
- **Probability / margin:** How strongly the model prefers the predicted class. How to read it: Probabilities near 1/k mean the model is guessing; many borderline samples = reconsider the model. Rule of thumb: Probabilities are model-dependent; calibrated sets (below) are more reliable.
- **Prediction interval (regression):** A range that should contain the true value with the chosen confidence (e.g. 95 %). How to read it: Wide intervals = low precision. Report value ± half-width. Rule of thumb: Valid only inside the calibrated range of y.
- **Prediction sets (classification, conformal):** A set of classes that contains the true class with the chosen confidence. How to read it: One class = confident; two or more = ambiguous; empty = nothing fits (unknown sample).
- **Applicability domain (T^2, Q, DD):** Does the new sample look like the data the model was trained on? How to read it: Outside the domain → the prediction is an extrapolation and should NOT be trusted. Rule of thumb: Do not report predictions of samples outside the domain; investigate or add similar samples to the training set.



15. Hyperparameters: defaults, data-based ranges and your own

A hyperparameter is a setting chosen before training (number of latent variables, C, gamma, number of trees…). The Hyperparameters tab of the Classification and Regression pages shows, for every algorithm, three possible search spaces and lets you edit them:

- **Library defaults:** the grid shipped with the app. Reasonable for a first look.

- **Recommended for your dataset:** ranges computed from the size and the characteristics of your data (n samples, p variables, classes, spread of y). The equation behind every range is shown, instantiated with your numbers.
- **Custom:** type your own list of values (for example 0.1, 1, 10) and the optimization uses exactly those.

Where n_fit is the number of samples available in each cross-validation fit, p the number of variables and n_train the number of training samples. The main rules are:

15.1 Rules used for the recommended ranges

> *PLS / PLS-DA components:* $k_max = min(20, p, n_fit/5) \quad \rightarrow$ *integers 1 … k_max*

At least about five samples per latent variable; above the optimum the model fits noise.

> *Logistic regression:* $C_0 = n_fit / p \quad grid = C_0 \times 10\text{^}k, \quad k = -2 \dots +2$
>
> *Ridge:* $\alpha_0 = p / n_fit \quad grid = \alpha_0 \times 10\text{^}k, \quad k = -3 \dots +2$
>
> *Lasso / Elastic Net:* $\alpha_max = max_j\ |x_j^T(y - \bar{y})| / n \quad grid = \alpha_max \times \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$

α_max is the smallest penalty that sets every coefficient to zero, so the grid goes from almost no shrinkage to total shrinkage. For Elastic Net, l1_ratio runs from 0.1 (Ridge-like, keeps groups of correlated variables) to 0.95 (Lasso-like, selects few variables).

> *SVM (RBF):* $\gamma_0 = 1 / (p \cdot Var(X)) \quad grid = \gamma_0 \times \{0.1, 0.3, 1, 3, 10\}; \quad C = 10^{-1} \dots 10^{3}$
>
> *SVR:* $C_0 = max(|\bar{y} + 3\sigma_y|, |\bar{y} - 3\sigma_y|) \quad grid = C_0 \times \{0.01, 0.1, 1, 10\}; \quad \varepsilon = \sigma_y \times \{0.01, 0.05, 0.1, 0.2\}$

The rule for C follows Cherkassky and Ma (2004); ε should be about the noise of the reference method.

> *KNN:* $k_0 \approx \sqrt{n_fit} \quad$ *odd values up to* $2 \cdot k_0 + 1$*, never above the smallest class in training*
>
> *Decision tree:* $d_max \approx log_2(n_train) + 2 \qquad$ *Random Forest:* $d_max \approx 2 \cdot log_2(n_train)$

Random Forest max_features: sqrt(p) and $log_2(p)$ of the variables; with hundreds of correlated variables (spectra) small fractions work best. The number of trees (100, 200, 300) only costs time.

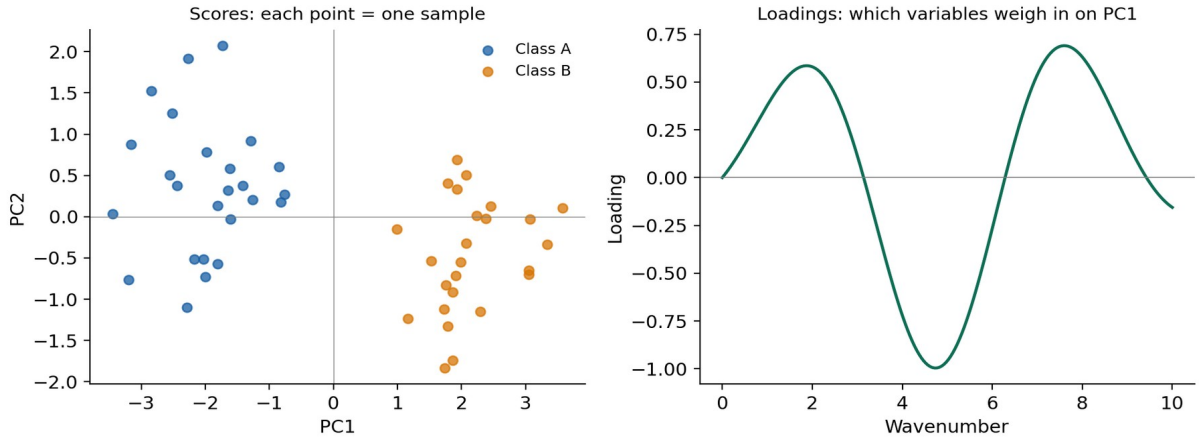
> *XGBoost:* $d_max = clip(\lfloor log_2(n_train/10) \rfloor + 1, 2, 8) \quad$ *learning rate {0.03, 0.1, 0.3} rounds {100, 200, 400}*
>
> *Neural network:* $h = clip(2 \cdot \sqrt{n_train}, 10, 100) \quad grid = (h), (2h), (h, h/2); \quad \alpha_0 = 1 / n_train$

With few samples use few neurons: a network with more weights than samples memorises them.

15.2 Good practice when editing ranges

- Make the grid wide first (powers of ten), then narrow it around the best value. If the best value sits at the edge of the grid, extend the grid in that direction.
- **Larger grids cost time:** the number of fits is the product of the lengths of all lists times the cross-validation folds. The app shows the size of the grid before you run it.

- Hyperparameters are always tuned inside the cross-validation of the training set. The test set is never used to choose them.



16. Public datasets to practise (and how to cite them)

The Public datasets tab lists tables and spectra that you can download legally to practise. Download the file, load it in Data and follow the learning path. Always keep the source and the citation in your notes (the final report has a field for it): the authors who shared their data deserve to be credited.

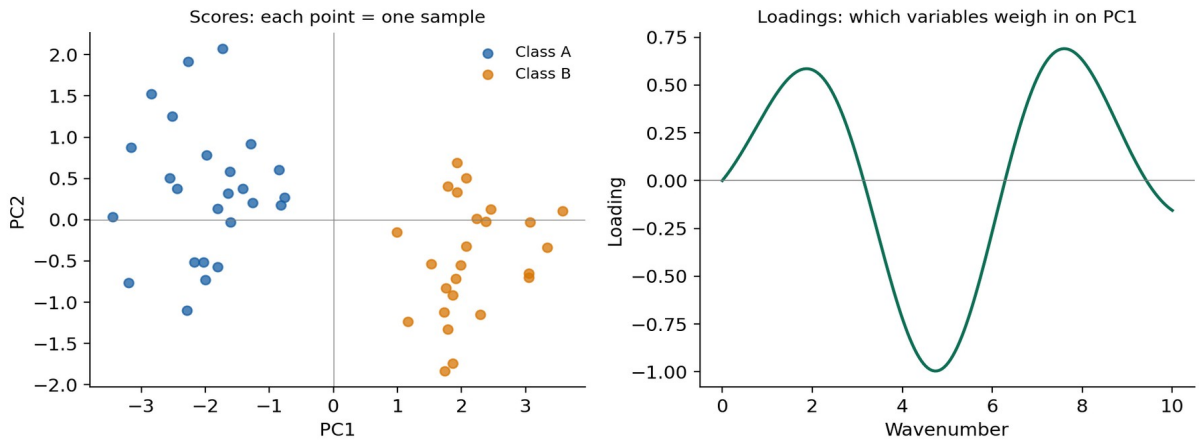
- Wine (178 wines, 13 chemical variables) [Table; Classification]. The classic chemometrics dataset: Italian wines from 3 cultivars analysed for alcohol, phenols, colour, proline… Perfect for PCA, LDA, PLS-DA and SIMCA. Where: https://archive.ics.uci.edu/dataset/109/wine. How to cite: Aeberhard, S. & Forina, M. (1992). Wine [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5PC7J — CC BY 4.0.
- **Iris (150 flowers, 4 measurements) [Table; Classification]. Fisher's iris data:** three species, four measurements. The smallest dataset to see classes that separate and classes that overlap. Where: https://archive.ics.uci.edu/dataset/53/iris. How to cite: Fisher, R. A. (1936). Iris [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C56C76 — CC BY 4.0.
- Diabetes (442 patients, 10 clinical variables) [Table; Regression]. Baseline clinical variables (age, BMI, blood pressure, six serum measurements) and disease progression one year later. Distributed with scikit-learn. Where: https://scikit-learn.org/stable/datasets/toy_dataset.html#diabetes-dataset. How to cite: Efron, B., Hastie, T., Johnstone, I. & Tibshirani, R. (2004). Least angle regression. Annals of Statistics, 32(2), 407–499.
- UCI Machine Learning Repository [Table; Classification · Regression]. Hundreds of curated datasets (many chemical/food/medical), each with its own page, DOI, licence and citation. Where: https://archive.ics.uci.edu/. How to cite: Use the 'Cite' box of each dataset page (it gives the DOI).
- Eigenvector Research — benchmark datasets [NIR · IR · process; Regression · Calibration transfer]. Corn NIR measured on three instruments (moisture, oil, protein, starch), the 2002 pharmaceutical-tablet 'shootout', diesel fuel NIR, NIR/IR images and more. A standard source for calibration and standardisation studies. Where: https://eigenvector.com/resources/data-sets/. How to cite: Each dataset page gives its acknowledgement/citation (e.g. Cargill for corn, SWRI for diesel, the IDRC 2002 shootout for tablets). Quote it.

- Tecator — NIR of meat (fat, water, protein) [NIR; Regression]. 215 minced-meat samples, 100-channel NIR transmittance (850–1050 nm), fat/water/protein by chemical analysis. Very widely used for benchmarking. Where: https://lib.stat.cmu.edu/datasets/tecator. How to cite: Follow the citation request of the StatLib page (Tecator, Borggaard & Thodberg).
- Gasoline NIR and octane number [NIR; Regression]. 60 gasoline samples, NIR 900–1700 nm, octane number. Included in the R package 'pls'. Where: https://cran.r-project.org/package=pls. How to cite: Kalivas, J. H. (1997). Two data sets of near infrared spectra. Chemometrics and Intelligent Laboratory Systems, 37, 255–259.
- Open Soil Spectral Library (OSSL) [NIR · Vis-NIR · MIR; Regression]. Large soil spectral libraries with laboratory reference values, open data and open tools (explorer, engine, manual). Where: https://soilspectroscopy.org/. How to cite: Follow the citation given in the OSSL manual (soilspectroscopy.github.io/ossl-manual).
- OliveOil — FT-MIR of extra-virgin olive oils (4 countries) [FT-MIR; Classification]. 120 spectra (570 points) of olive oils from Greece, Italy, Portugal and Spain. A classic authentication problem: PCA, SIMCA and PLS-DA. Where: https://www.timeseriesclassification.com/description.php?Dataset=OliveOil. How to cite: Tapp, H. S., Defernez, M. & Kemsley, E. K. (2003). FTIR spectroscopy and multivariate analysis can distinguish the geographic origin of extra virgin olive oils. J. Agric. Food Chem., 51, 6110–6115.
- Coffee — FT-IR of instant coffee (Arabica vs Robusta) [FT-MIR; Classification]. 56 FT-IR spectra (286 points) to distinguish Arabica from Robusta. Where: https://www.timeseriesclassification.com/description.php?Dataset=Coffee. How to cite: Briandet, R., Kemsley, E. K. & Wilson, R. H. (1996). Discrimination of Arabica and Robusta in instant coffee by Fourier transform infrared spectroscopy and chemometrics. J. Agric. Food Chem., 44(1), 170–174.
- Olive and vegetable oils — FTIR-ATR and Raman (Zenodo) [FT-MIR · Raman; Classification · Regression]. Raw spectra (CSV) of olive oil, sunflower oil and blends, to quantify adulteration. Where: https://zenodo.org/doi/10.5281/zenodo.14651815. How to cite: Cite the Zenodo record (DOI 10.5281/zenodo.14651815) and the associated paper (LWT – Food Science and Technology, 86, 174–184, 2017).
- NIST Chemistry WebBook — reference IR and mass spectra [IR · MS; Identification]. Reference spectra of thousands of compounds (gas-phase IR, mass spectra, quantitative IR database). Where: https://webbook.nist.gov/chemistry/. How to cite: Use the Citation Guide of the site (database DOI 10.18434/T4D303).
- SDBS — Spectral Database for Organic Compounds (AIST, Japan) [NMR · IR · Raman · MS; Identification]. Free spectra of organic compounds (^{1}H and ^{13}C NMR, IR, Raman, MS). Useful to assign bands and peaks. Where: https://sdbs.db.aist.go.jp/. How to cite: Read the terms of use of the site and credit AIST/SDBS.
- USGS Spectral Library [Vis-NIR-SWIR; Identification]. Laboratory reflectance spectra of minerals, vegetation and man-made materials. Where: https://www.usgs.gov/labs/spectroscopy-lab/science/spectral-library. How to cite: See the citation recommended on the library page.
- RRUFF — Raman, XRD and chemistry of minerals [Raman; Identification]. Integrated database of well-characterised mineral Raman spectra (many laser wavelengths), freely downloadable. Where: https://rruff.info/. How to cite: Check the terms on the site and cite Lafuente, B. et al. (2015), The power of databases: the RRUFF project, in Highlights in Mineralogical Crystallography.
- Bacteria-ID — Raman spectra of pathogenic bacteria [Raman; Classification]. Thousands of Raman spectra of 30 bacterial and yeast isolates (code on GitHub, data on a linked Dropbox folder). Where: https://github.com/csho33/bacteria-ID. How to cite: Ho, C.-S. et al. (2019). Rapid identification of

pathogenic bacteria using Raman spectroscopy and deep learning. Nature Communications, 10, 4927. Code: MIT licence.

- Open Specy — Raman and FTIR reference spectra (microplastics) [Raman · FTIR; Identification]. Open-source tool and reference libraries to process, identify and share Raman and FTIR spectra. Where: https://openspecy.org/. How to cite: Cowger, W. et al. (2025). Open Specy 1.0. Analytical Chemistry, 97(32), 17345–17356. https://doi.org/10.1021/acs.analchem.5c00962
- MetaboLights (EMBL-EBI) [NMR · MS · GC/LC-MS; Classification · Regression]. The reference repository of metabolomics studies, with raw and processed NMR and MS data and rich sample metadata. Search by organism, matrix or technique. Where: https://www.ebi.ac.uk/metabolights/. How to cite: Cite the study accession (MTBLS…) and the MetaboLights paper (Yurekten, O. et al., Nucleic Acids Res., 2024).
- Metabolomics Workbench (NIH) [NMR · GC-MS · LC-MS; Classification · Regression]. More than 2 000 public studies (NMR, GC-MS, LC-MS) with downloadable data and analysis tools. Where: https://www.metabolomicsworkbench.org/. How to cite: Cite the study ID (ST…) and the Workbench (Sud, M. et al., Nucleic Acids Res., 2016).
- Maize grains — NMR and LC-MS (Zenodo) [NMR · LC-MS; Classification]. ^{1}H NMR (and CPMG) spectra of maize grain and pellet extracts, plus LC-MS raw files. Large (7.8 GB): download only the NMR folders. Where: https://doi.org/10.5281/zenodo.1097409. How to cite: Bernillon, S. et al. (2017). Maize grains and maize grain-based diets dataset. Zenodo. https://doi.org/10.5281/zenodo.1097409 — CC BY-NC 4.0 (non-commercial).
- Zenodo [Any; Any]. General open repository (CERN). Search for 'NIR spectra', 'Raman dataset', 'chromatogram', 'chemometrics'. Every record has a DOI. Where: https://zenodo.org/. How to cite: Cite the record's DOI and respect its licence.
- Mendeley Data [Any; Any]. Open repository where many analytical-chemistry papers deposit their spectral datasets. Where: https://data.mendeley.com/. How to cite: Cite the dataset DOI and the paper.
- Figshare [Any; Any]. General repository used by many journals for supplementary data. Where: https://figshare.com/. How to cite: Cite the DOI.
- **Dryad [Any; Any]. Curated repository for data underlying published papers. Where:** https://datadryad.org/. How to cite: Cite the DOI.
- OpenML [Table; Classification · Regression]. Machine-learning datasets with standard tasks and benchmarks. Where: https://www.openml.org/. How to cite: Cite the dataset and the original authors.
- **Kaggle Datasets [Any; Any]. Community datasets (quality varies:** check the original source and licence). Where: https://www.kaggle.com/datasets. How to cite: Cite the original authors, not only the Kaggle page.
- Google Dataset Search [Any; Any]. A search engine for datasets across repositories. Where: https://datasetsearch.research.google.com/. How to cite: Cite the original repository record.

Licences and terms change. Check the page of the dataset before reusing it, and cite the version you used.



Putting it all together

This guide accompanies Espectrometrika Academy. Every concept explained here has a matching control in the app: choosing a preprocessing step, viewing PCA and its loadings, cutting a dendrogram, screening models, training a classification or regression model, building a SIMCA class model, reading a Williams plot or a learning curve, comparing models statistically, editing hyperparameter ranges. The best way to internalise these concepts is to try them on real data, watching how the results change as each setting is adjusted, and to come back here whenever a plot or a metric needs a second look.

The app is organised in groups: Home · Data & preprocessing (Data, Preprocessing, Crop) · Exploratory (PCA, Outliers, Dendrogram, Other tools including t-SNE, UMAP and MCR-ALS) · Models & prediction (Model screening, Classification, SIMCA, Regression, Prediction with reliability tools, Hyperparameters) · Final report · Learn (Glossary and cheat sheet, Public datasets) · Contact. Every control carries a short help text, and nothing heavy runs until you click its button. The language of the interface can be switched between English and Castellano at the top of the page; this guide and the PDF reports stay in English.

Contact

Questions about chemometrics, help with your own data, a meeting, a course or a collaboration are welcome: Pablo Facundo Sánchez. Use the Contact tab of the app or write to espectrometrika@gmail.com. Professional advice for building your own models (calibration, classification, validation, training for your team) is also available. Please do not send confidential or personal data by e-mail.